Large Language Model for Verilog Code Generation: Literature Review and the Road Ahead

GUANG YANG, Zhejiang University, China and Northwestern Polytechnical University, China
WEI ZHENG, Northwestern Polytechnical University, China
DONG LIANG, Northwestern Polytechnical University, China
PENG HU, Northwestern Polytechnical University, China
YUKUI YANG, Northwestern Polytechnical University, China
SHAOHUA PENG, Northwestern Polytechnical University, China
ZHENGHAN LI, Northwestern Polytechnical University, China
JIAHUI FENG, Northwestern Polytechnical University, China
XIAO WEI, Northwestern Polytechnical University, China
KEXIN SUN, Northwestern Polytechnical University, China
DEYUAN MA, Northwestern Polytechnical University, China
HAOTIAN CHENG, Northwestern Polytechnical University, China
YIHENG SHEN, Nantong Normal College, China
XIANG CHEN, Nantong University, China
XING HU, Zhejiang University, China
TERRY YUE ZHUO, Monash University, Australia
DAVID LO, Singapore Management University, Singapore

Code generation represents a critical intersection of Software Engineering (SE) and Artificial Intelligence (AI). Within this broader landscape, Verilog, as a representative hardware description language (HDL), is fundamental to Electronic Design Automation (EDA), recent research has increasingly focused on leveraging

This work was partially supported by the National Natural Science Foundation of China (NSFC, No. 62141208). Guang Yang is also supported by the Postdoctoral Fellowship Program of CPSF under Grant Number GZC20260902.
Authors' Contact Information: Guang Yang, Zhejiang University, Hangzhou, Zhejiang, China and Northwestern Polytechnical University, Xian, Shaanxi, China; e-mail: novelyg@outlook.com; Wei Zheng (corresponding author), Northwestern Polytechnical University, Xian, Shaanxi, China; e-mail: wzheng@nwpu.edu.cn; Dong Liang, Northwestern Polytechnical University, Xian, Shaanxi, China; e-mail: rwliangdong@mail.nwpu.edu.cn; Peng Hu, Northwestern Polytechnical University, Xian, Shaanxi, China; e-mail: 2136197200@qq.com; Yukui Yang, Northwestern Polytechnical University, Xian, Shaanxi, China; e-mail: 1572365268@qq.com; Shaohua Peng, Northwestern Polytechnical University, Xian, Shaanxi, China; e-mail: 2020302993@mail.nwpu.edu.cn; Zhenghan Li, Northwestern Polytechnical University, Xian, Shaanxi, China; e-mail: edwardlee_02@outlook.com; Jiahui Feng, Northwestern Polytechnical University, Xian, Shaanxi, China; e-mail: fengjiahui@mail.nwpu.edu.cn; Xiao Wei, Northwestern Polytechnical University, Xian, Shaanxi, China; e-mail: 1376463051@qq.com; Kexin Sun, Northwestern Polytechnical University, Xian, Shaanxi, China; e-mail: 2522405995@qq.com; Deyuan Ma, Northwestern Polytechnical University, Xian, Shaanxi, China; e-mail: madeyuan@mail.nwpu.edu.cn; Haotian Cheng, Northwestern Polytechnical University, Xian, Shaanxi, China; e-mail: cht_sd@163.com; Yiheng Shen, Nantong Normal College, Nantong, Jiangsu, China; e-mail: yiheng.s@ntnc.edu.cn; Xiang Chen, Nantong University, Nantong, Jiangsu, China; e-mail: xchencs@ntu.edu.cn; Xing Hu, Zhejiang University, Hangzhou, China; e-mail: xinghu@zju.edu.cn; Terry Yue Zhuo, Monash University, Melbourne, Australia; e-mail: terry.zhuo@monash.edu; David Lo, Singapore Management University, Singapore, Singapore; e-mail: davidlo@smu.edu.sg.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 0360-0300/2026/12-ART411
https://doi.org/10.1145/3841636

Large Language Models (LLMs) to automate Verilog code generation, particularly at the Register Transfer Level (RTL) design. Despite growing interest, a comprehensive survey of this domain remains absent. This review addresses this gap by providing a systematic literature review of LLM-based Verilog code generation, analyzing 102 papers (70 published and 32 high-quality preprints) from SE, AI, and EDA venues. We structure our analysis around four key research questions: (1) identifying the LLMs utilized, (2) examining evaluation datasets and metrics, (3) categorizing generation techniques, and (4) analyzing alignment approaches. Furthermore, we synthesize findings to identify critical limitations in current studies regarding effectiveness and integration. Finally, we outline a roadmap highlighting potential opportunities for future research in LLM-assisted hardware design.

CCS Concepts: • **Software and its engineering → Domain specific languages**; • **Hardware → Electronic design automation**; • **Computing methodologies → Artificial intelligence**.

Additional Key Words and Phrases: Literature review, verilog code generation, large language models

ACM Reference Format:
Guang Yang, Wei Zheng, Dong Liang, Peng Hu, Yukui Yang, Shaohua Peng, Zhenghan Li, Jiahui Feng, Xiao Wei, Kexin Sun, Deyuan Ma, Haotian Cheng, Yiheng Shen, Xiang Chen, Xing Hu, Terry Yue Zhuo, and David Lo. 2026. Large Language Model for Verilog Code Generation: Literature Review and the Road Ahead. *ACM Comput. Surv.* 58, 16, Article 411 (December 2026), 35 pages. https://doi.org/10.1145/3841636

1 Introduction

Large Language Models (**LLMs**) have revolutionized code generation, with models like Codex [29] and CodeLlama [94] demonstrating exceptional proficiency in general-purpose languages [86]. This success has rapidly extended to specialized domains, including scientific computing [138], web development [44], and domain-specific languages [59]. However, hardware design presents distinct challenges compared to software development. The EDA industry faces widening productivity gaps due to escalating circuit complexity [122] and the specialized expertise required for HDLs like Verilog (e.g., timing and architecture). These factors, combined with labor-intensive manual workflows and severe talent shortages, create significant bottlenecks in modern hardware development.

Research on LLM-based Verilog generation has expanded dramatically, from a single study in 2020 [84] to 66 papers in 2025 (see Figure 4). This research spans AI, EDA, and interdisciplinary venues, leading to fragmented knowledge with inconsistent terminologies, evaluation metrics, and optimization approaches. Despite this growth, the lack of a systematic review hinders the community from unifying these contributions, establishing best practices, and identifying future directions.

To address this gap, we conducted a **Systematic Literature Review** (**SLR**) [60] using the **Quasi-Gold Standard** (**QGS**) strategy [133]. As shown in Figure 3, we combined manual searches of premier venues (e.g., AAAI, DAC) with automated searches across six major databases (e.g., IEEE Xplore, arXiv). Following a rigorous three-stage filtering and snowballing process, we selected 102 high-quality papers (70 peer-reviewed, 32 preprints) from 2020 to October 2025. This review answers four key research questions concerning the employed LLMs (RQ1), datasets and metrics (RQ2), optimization techniques (RQ3), and alignment approaches (RQ4).

Related Literature Reviews. While several surveys have examined LLM applications in code generation, software engineering, and hardware design automation, our work represents the first comprehensive review focusing specifically on Verilog code generation. Table 1 summarizes how our work differs from existing literature reviews. Prior surveys fall into two primary categories:

Table 1. Comparison with Related Literature Reviews

Survey	Year	Papers	Topics	Verilog Focus
General Code Generation Surveys				
Jiang et al. [57]	2024	235	General Programming Languages	✗
Joel et al. [59]	2024	111	Low-Resource and Domain-Specific Programming Languages	Partial
Chen et al. [30]	2025	601	Deep Learning-based Software Engineering	✗
EDA-focused Surveys				
Fang et al. [39]	2025	250	Layout/Netlists/HDL/Assertion Generation	Partial
He et al. [51]	2024	211	EDA Generation, Verification, and Debugging	Partial
Chen et al. [28]	2024	≈ 200	A Paradigm Shift from AI4EDA toward AI-rooted EDA (full-workflow AI4EDA)	Partial
Our Verilog-specific Survey				
Our work	**2025**	**102**	Verilog Code Generation	✓

(1) *general code generation surveys* that predominantly concentrate on high-level programming languages like Python and Java but neglect hardware description languages, and (2) *broad EDA surveys* that cover multiple aspects of hardware design automation without in-depth analysis of LLM-based Verilog generation.

Existing surveys on general code generation [53, 57, 59] overlook hardware-specific challenges like timing and synthesizability. Similarly, broader EDA surveys [28, 39, 51] touch upon RTL generation but lack systematic analysis of Verilog-specific datasets and methodologies. In contrast, our survey provides the first comprehensive review dedicated to LLM-based Verilog generation, offering a systematic analysis of models, datasets, evaluation metrics, and alignment techniques from both AI and hardware perspectives.

In general, this study makes the following **contributions**:

- We present the first systematic literature review on LLM-based Verilog code generation, analyzing 102 papers (70 peer-reviewed, 32 high-quality preprints) spanning 2020-2025.
- We provide comprehensive taxonomy and trend analysis of LLMs for Verilog generation (RQ1), systematically analyze dataset construction and evaluation evolution across 28 benchmarks and 34 training datasets (RQ2), investigate adaptation and optimization techniques (RQ3), and examine alignment techniques addressing human-centric requirements including security, efficiency, copyright, and hallucinations (RQ4).
- We identify key research limitations and propose a comprehensive roadmap for future directions in LLM-assisted hardware design.

Survey Structure. Figure 1 illustrates the organization of this survey. Sections 2 and 3 present the background and review methodology. The core analysis follows in Sections 4 to 7, which respectively examine the used LLMs, datasets and metrics, generation techniques, and alignment strategies. Finally, Sections 8 and 9 discuss open challenges and future roadmaps, followed by the conclusion in Section 10.

2 Background and Preliminaries

As shown in Figure 2, the EDA workflow begins with *Architectural Specification* (Phase 1) and *RTL Design* (Phase 2), where HDLs like Verilog describe hardware architecture. *Functional Verification* (Phase 3) then validates correctness via testbenches, forming an iterative loop with RTL design. Subsequent phases include *Logic Synthesis* (Phase 4), *Place and Route* (Phase 5), *Physical Verification* (Phase 6), and finally *Fabrication Preparation* (Phase 7).

Verilog code generation (dashed box in Figure 2) primarily targets the first three phases. The quality of generated code is critical, directly impacting downstream synthesis, timing, and power metrics. This task differs fundamentally from software generation due to:

- **Concurrency:** Hardware is inherently parallel, unlike sequential software.
- **Timing Constraints:** Designs must satisfy strict setup/hold times and delays.

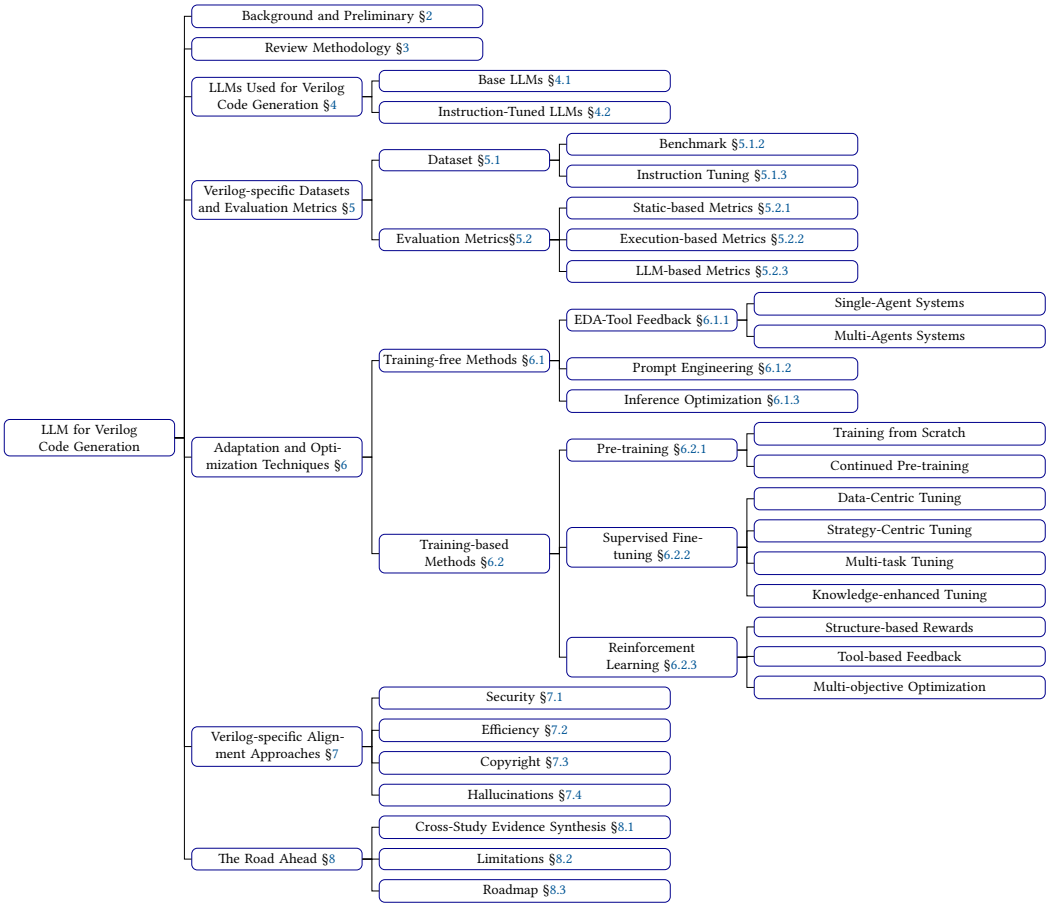


Fig. 1. Structure of this survey.

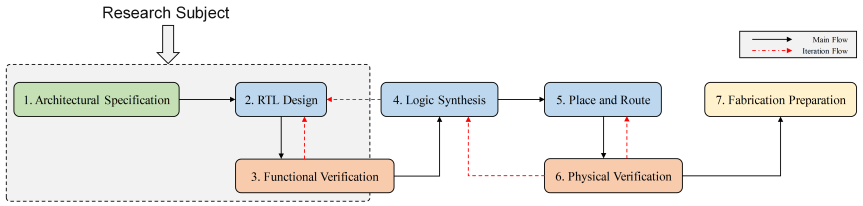


Fig. 2. The EDA workflow for integrated circuit development.

— **Physical Limitations:** Code must respect area, power, and routing constraints.
— **Synthesizability:** Verification requires ensuring both functional and timing correctness.
— **Domain Expertise:** It demands deep knowledge of digital logic and architecture.

These characteristics highlight the unique challenges of applying LLMs to hardware design.

Formally, LLM-based Verilog generation is a mapping $f_\theta : (D, I) \rightarrow V$, where θ are model parameters, D is the natural language specification (functional and constraint requirements), I

represents optional multimodal inputs (e.g., diagrams), and V is the generated code. Most current approaches simplify this to text-only generation $f_\theta : D \to V$.

Correctness is evaluated via a function $verify : (V, S, T) \to \{pass, fail\} \times M$, where S is the reference specification and T denotes testbenches. This process typically includes: (1) *syntactic validation* (compilability); (2) *functional correctness* (simulation where $\forall t \in T : sim(V, t) = expected(t)$); (3) *semantic similarity* (AST or text metrics); and (4) *formal equivalence* (mathematical proof against reference designs).

3 Review Methodology

Following the guide by Kitchenham et al. [60], which is used in most other SLRs [53, 54], our methodology is organized as follows: planning the review (i.e., Section 3.1, 3.2), conducting the review (i.e., Section 3.3), and analyzing the basic review results (i.e., Section 3.4).

3.1 Research Question

This study thus aims at answering the following research questions:

RQ1: What LLMs have been employed for Verilog code generation? This question establishes a taxonomy of model architectures (Base vs. Instruction-Tuned LLMs, open vs. closed-source), revealing adoption trends and the evolution of model preferences in this domain.

RQ2: How are Verilog-specific datasets constructed and evaluated? We investigate dataset methodologies and evaluation metrics, which are critical for interpreting model performance, ensuring benchmark validity, and guiding future dataset optimization.

RQ3: What adaptation and optimization techniques are applied? This RQ examines strategies for customizing general-purpose LLMs to Verilog's unique constraints, identifying effective methods to enhance domain-specific performance.

RQ4: What alignment techniques address human-centric requirements? We analyze alignment approaches ensuring generated code conforms to human intent, addressing critical risks such as security vulnerabilities, copyright infringement, and trustworthiness.

3.2 Search Strategy

As shown in Figure 3, we employed the "QGS" [133] approach, combining manual identification of key studies with automated string-based searches and snowballing techniques. Rigorous filtering was then applied to select relevant papers. Our search targets publications from 2020 (the field's inception [84]) to 2025 to ensure comprehensive coverage.

3.2.1 Search Items. During the manual search phase, we selected eight premier conferences and journals (i.e., AAAI, ACL, ICML, ICLR, NeurIPS, DAC, MLCAD, and TCAD), and systematically retrieved papers applying LLMs to Verilog code generation tasks. We identified 16 papers (14 papers from conferences and 2 papers from journals) that aligned with our research objectives. These 16 relevant publications constitute the foundational corpus for constructing our QGS, serving as the basis for extracting comprehensive search terms and validating our automated search strategy. Our search string combines keywords for "Verilog code generation" and "LLMs", requiring matches from both categories to ensure precision. The comprehensive search keyword sets are as follows:

- **Keywords related to Verilog code generation (12 terms):** Verilog, HDL, Hardware Description Language, RTL, Register Transfer Level, Digital Design, Hardware Design, Electronic Design Automation, EDA, Verilog Generation, FPGA, ASIC
- **Keywords related to LLMs (13 terms):** LLM, Large Language Model, Language Model, GPT, ChatGPT, Transformer, fine-tuning, prompt engineering, In-context learning, Natural Language Processing, NLP, Machine Learning, AI

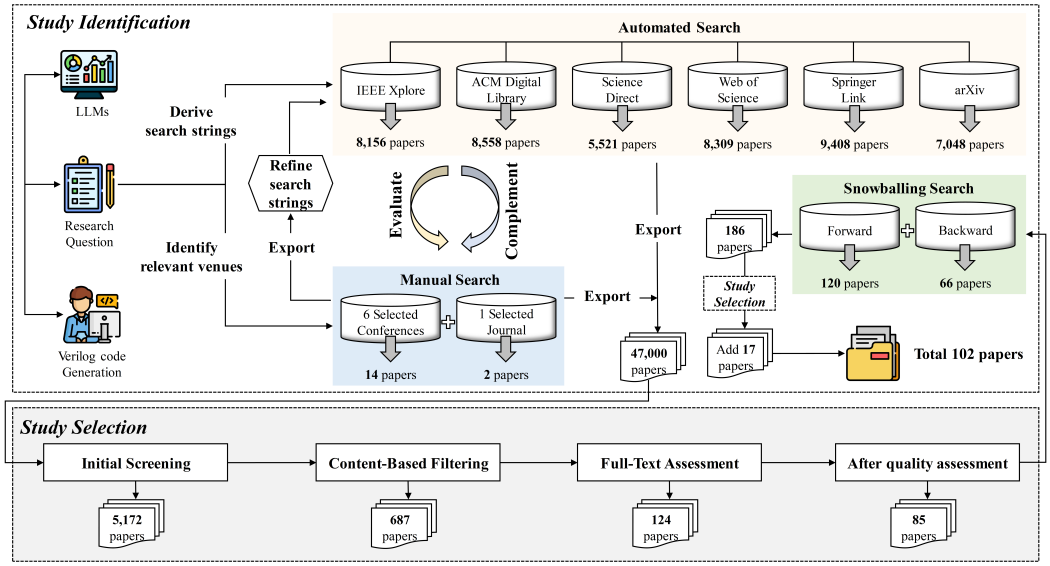


Fig. 3. Systematic literature review process for study identification and selection.

3.2.2 Search Databases. Following the establishment of our search strings, we conducted systematic automated searches across six widely recognized academic databases: IEEE Xplore [6], ACM Digital Library [1], ScienceDirect [9], Web of Science [11], SpringerLink [10], and arXiv [3]. Consistent with our temporal scope targeting publications from 2020 onward, we applied appropriate date filters to each database search.

3.3 Search Selection

3.3.1 Inclusion and Exclusion Criteria. To ensure methodological rigor, we established inclusion and exclusion criteria (Table 2) based on established SLR practices [13]. These criteria ensure selected studies directly address our research questions.[1]

Our paper selection process employed a three-stage filtering approach to systematically reduce the candidate pool: ① **Stage 1 (Initial Screening):** Exclusion of short papers (< 5 pages) and duplicates (criteria 1, 6) reduced the corpus to 5,172 papers. ② **Stage 2 (Content-Based Filtering):** Manual screening of venues, titles, and abstracts narrowed selection to 687 papers. We retained high-quality preprints but excluded non-peer-reviewed materials like theses and reports (criteria 2-3). ③ **Stage 3 (Full-Text Assessment):** Full-text review excluded studies focused solely on repair/testing without generation (criterion 5) or lacking implementation (criterion 4). This yielded 124 primary studies.

3.3.2 Quality Assessment. To ensure study quality and transparency, we implemented a rigorous assessment framework comprising five **Quality Assessment Criteria (QACs)** (Table 3) [128]. Each criterion was evaluated on a 0-3 scale, with a total inclusion threshold set at 12 points (80% of the maximum). For preprints lacking venue scores (i.e., QAC1 is 0), they were retained only if they demonstrated exceptional merit in other dimensions to meet the threshold.

[1]For preprint papers released on arXiv, we performed manual quality assessments to determine their eligibility for inclusion based on methodological soundness and contribution relevance.

Table 2. Inclusion Criteria and Exclusion Criteria

Inclusion criteria	
1)	The paper claims that an LLM is used.
2)	The paper claims that the study involves to solve Verilog code generation.
3)	The paper with accessible full text and must be written in English.
Exclusion criteria	
1)	Short papers whose number of pages is less than 5.
2)	Books, records, theses, tool demos papers, editorials, or venues not subject to a full peer-review process.
3)	The paper is a literature review or survey.
4)	The paper mentions LLMs only in future work or discussions rather than using LLMs in the approach.
5)	The paper does not involve Verilog code generation.
6)	Duplicate papers or similar studies authored by the same authors.

Table 3. Checklist of Quality Assessment Criteria (QAC)

ID	Quality Assessment Criteria
QAC1	Was the study published in a prestigious (e.g., CCF ranking) venue?
QAC2	Does the study make a contribution to the academic or industrial community?
QAC3	Does the study provide a clear description of the workflow and implementation of the proposed approach?
QAC4	Are the experiment details, including datasets, baselines, and evaluation metrics, clearly outlined?
QAC5	Do the findings from the experiments strongly support the main arguments presented in the study?

This process yielded a final corpus of 85 studies, consisting of 55 peer-reviewed publications and 30 high-quality preprints, balancing scholarly rigor with the inclusion of emerging research.

3.3.3 Forward and Backward Snowballing. To ensure comprehensive coverage, we conducted systematic forward and backward snowballing [120]. This supplementary strategy involves examining the reference lists of our primary studies (backward snowballing) to find missed relevant works, and utilizing citation databases to identify subsequent publications that cite our selected studies (forward snowballing), thereby capturing emerging developments.

The snowballing process initially yielded 186 additional candidates. These papers were subjected to our established selection pipeline, including screening against inclusion/exclusion criteria and quality assessment. Following this rigorous evaluation, we identified 15 additional relevant studies. This resulted in a final corpus of 102 papers (70 peer-reviewed publications and 32 high-quality preprints), which forms the empirical foundation of our review.

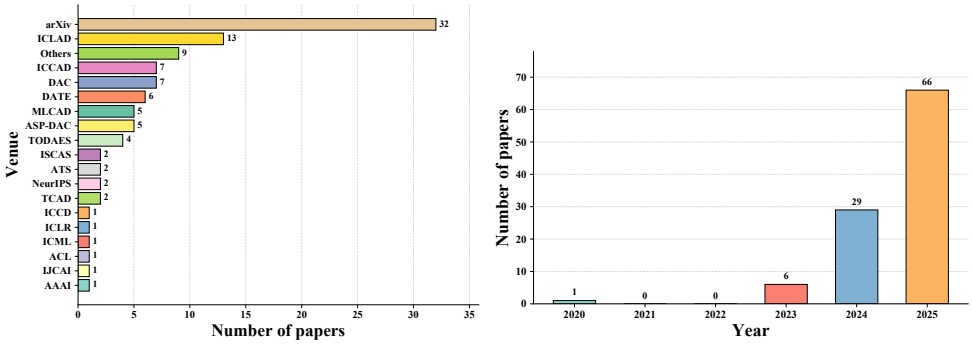
3.4 Data Extraction and Analysis

Figure 4 presents the distribution of the 102 selected papers and the temporal growth. From a single study in 2020 and a dormant period (2021-2022), publications surged to 6 in 2023, 29 in 2024, and 66 by September 2025. This exponential trajectory underscores the intensifying research interest in this domain.

4 RQ1: LLMs Employed for Verilog Code Generation

Through our systematic literature review of 102 included studies, we identify that researchers have employed a diverse range of LLMs for Verilog code generation tasks.

Statistical Scope. Our analysis encompasses all LLMs utilized in the reviewed papers, whether for *method implementation* or as *baseline comparisons*, ensuring comprehensive coverage of the LLM adoption landscape in Verilog code generation research. Based on their adaptation for Verilog tasks, we categorize these LLMs into two main types: **Base LLMs** (*general-purpose models without Verilog-specific adaptation*) and **IT LLMs** (*Instruction-Tuned models with Verilog-specific adaptation*).



(a) Distribution of papers across venues. (b) Distribution of papers over years.

Fig. 4. Overview of the selected 102 papers' distribution.

Table 4. Usage of Open and Closed-Source LLM Families in Verilog Code Generation Studies (2023–2025)

Type	LLM Family	2023	2024	2025	Total
Open-source	Llama series	0	15	49	64
Open-source	DeepSeek series	0	6	48	54
Open-source	Qwen series	0	3	25	28
Open-source	CodeGen series	3	6	3	12
Open-source	StarCoder series	0	5	6	11
Open-source	Others	2	11	22	35
Open-source	**Subtotal**	**5**	**46**	**153**	**204**
Closed-source	GPT series	7	42	100	149
Closed-source	Claude series	0	5	14	19
Closed-source	Gemini	0	1	4	5
Closed-source	Others	0	3	3	6
Closed-source	**Subtotal**	**7**	**51**	**121**	**179**
Overall		**12**	**97**	**274**	**383**

4.1 Analysis of Base LLMs

As summarized in Table 4, Base LLMs account for 383 total usages across all surveyed studies and thus constitute the dominant paradigm for Verilog code generation. Although these models are not explicitly designed for hardware description languages, their strong general-purpose programming capabilities already support competitive Verilog understanding and generation.

4.1.1 Open-Source Base LLMs. Open-source Base LLMs exhibit substantial diversity and adoption, reaching 204 usages (53.3% of all Base LLM deployments). The landscape is structured around several major families:

(1) Llama Series. The Llama ecosystem [104] is the most widely adopted open-source family, with 64 total mentions. This reflects a clear preference for large, high-quality foundation models (e.g., CodeLlama, Llama3.x [94, 99]) that provide strong general code-generation capabilities rather than Verilog-specific specialization.

(2) DeepSeek Series. DeepSeek models [20] achieve 54 mentions, ranking second among open-source families. The use of both code-centric and reasoning-oriented variants (e.g., DeepSeek-Coder, DeepSeek-R1 [47, 48]) suggests growing interest in leveraging advanced reasoning for complex Verilog design and debugging.

(3) Qwen Series. The Qwen family accumulates 28 mentions, emerging as a competitive open-source alternative. Models such as CodeQwen and Qwen2.5-Coder [43, 126] strengthen code understanding and multi-language support, which is beneficial for heterogeneous hardware design toolchains.

(4) Others. Traditional code-oriented families such as CodeGen (12 mentions) [82, 83] and StarCoder (11 mentions) [62, 73] remain in use but are increasingly overshadowed by newer foundation families with larger scales and more diverse training data, indicating that general capability and ecosystem maturity often outweigh narrowly targeted code training. A substantial portion (35 mentions) corresponds to various other open-source models (e.g., CodeGemma [136], CodeT5+ [114], Phi [45, 63]), highlighting the experimental nature of the field and researchers' willingness to explore alternative architectures and training paradigms.

4.1.2 Closed-Source Base LLMs. Closed-source Base LLMs also exhibit strong adoption, with 179 usages (46.7% of all Base LLM deployments), predominantly provided by major technology companies.

(1) GPT Series. OpenAI's GPT series [7] dominates the closed-source segment with 149 mentions (83.2% of closed-source usage). Different generations (e.g., GPT-3.5, GPT-4, GPT-4o) are widely employed for Verilog code generation, while newer reasoning-oriented models (such as GPT-o1 and GPT-o3) are increasingly explored for more challenging design and verification tasks.

(2) Other Models. Anthropic's Claude family [2] reaches 19 mentions, positioning itself as a strong competitor in closed-source Verilog applications. Google's Gemini [4] appears as a more recent entrant with 5 mentions, indicating emerging interest but still limited deployment compared to GPT-series models.

4.1.3 Trends Analysis. The field exhibits rapid expansion, with total Base LLM usages growing from 12 in 2023 to 274 in 2025 (2,183% growth). This trajectory reflects both the fast-paced evolution of LLM technology and the community's increasing confidence in applying these models to hardware design.

We first analyze adoption trends from the perspective of Open-Source vs. Closed-Source models and then discuss architectural preferences:

(1) Open-Source vs. Closed-Source. Both categories grow rapidly, but with different trajectories and ecosystem dynamics:

- **Open-Source LLMs:** Usages increase from 5 to 46 to 153 mentions, corresponding to a 233% growth from 2024 to 2025. The evolution shows a shift from traditional code models to large foundation families:
  - **2023:** Dominated by classic code generators (e.g., CodeGen with 3 mentions).
  - **2024:** Transition phase where foundation families (Llama: 15, DeepSeek: 6, Qwen: 3 mentions) begin to dominate.
  - **2025:** Widespread adoption of the latest foundation models (Llama: 49, DeepSeek: 48, Qwen: 25 mentions), which become the de-facto standard for open-source Verilog generation.
- **Closed-Source LLMs:** Usages grow from 7 to 51 to 121 mentions (137% growth from 2024 to 2025), reflecting both consolidation and diversification:

Table 5. Instruction-Tuned LLMs for Verilog Code Generation (Classified by Model Weight Availability)

Verilog-Specific LLM	Foundation Model	Weight	URL
Closed-Source (Model Weights)			
AutoVCoder [41]	CL/CQ/DS-Coder	Closed	-
BetterV [85]	CL/CQ/DS-Coder	Closed	-
CodeGen-Verilog [66]	CodeGen	Closed	-
CraftRTL [67]	CL/DS-Coder/StarCoder2	Closed	-
DeepRTL [71]	CodeT5+	Closed	-
DeepRTL2 [72]	Llama3.1/DS-Coder	Closed	-
FreeV [22]	Llama3.1	Closed	-
ITERTL [124]	DS-Coder/DS-Coder-v2	Closed	-
MEV-LLM [80]	CodeGen/Gemma	Closed	-
OpenRTLSet [110]	Qwen2.5	Closed	-
PyraNet [78]	CL/DS-Coder	Closed	-
RTL++ [15]	CL	Closed	-
RTLRepoCoder [123]	DS-Coder	Closed	-
ScaleRTL [37]	DS-R1-Distill-Qwen	Closed	-
Veritas [93]	Llama-3.2	Closed	-
Open-Source (Model Weights)			
DAVE [84]	GPT-2	Open	https://shorturl.asia/VWPTn
ChipGPT [26]	Llama2/Llama3	Open	https://www.modelscope.cn/profile/changkaiyan
CodeV [139]	CQ/DS-Coder/QC	Open	https://huggingface.co/zhuyaoyu
CodeV-R1 [142]	QC	Open	https://huggingface.co/zhuyaoyu
GEMMV [132]	DS-R1-Distill-Qwen/Llama3.1	Open	https://huggingface.co/bxsk2024
Hardware Phi-1.5B [40]	Phi-1.5B	Open	https://huggingface.co/KSU-HW-SEC/Hardware_Phi_30k_version
HAVEN [129]	CL/DS-Coder/CQ	Open	https://huggingface.co/yangyiyao
MG-Verilog [135]	CL	Open	https://shorturl.asia/rDNSm
Mistral-Verilog [42]	Mistral	Open	https://huggingface.co/emilgoh/mistral-verilog
Origen [32]	DS-Coder	Open	https://huggingface.co/henryen/OriGen
ReasoningV [91]	QC	Open	https://modelscope.cn/models/GipsyAI/ReasoningV-7B
RTLCoder [68]	Mistral/DS-Coder	Open	https://github.com/hkust-zhiyao/RTL-Coder
VeriCoder [118]	QC	Open	https://huggingface.co/LLM4Code/VeriCoder_Qwen14B
VeriGen [101]	CodeGen	Open	https://huggingface.co/shailja
VeriLogos [77]	DS-Coder	Open	https://huggingface.co/97kjmin/VeriLogos
VeriPrefer [112]	Mistral/CL/DS-Coder/CQ/QC	Open	https://shorturl.asia/sMj3J
VeriReason [115]	QC/CL	Open	https://shorturl.asia/yaNkf
VeriSeek [113]	DS-Coder	Open	https://huggingface.co/LLM-EDA/VeriSeek
VeriThoughts [131]	QC	Open	https://shorturl.asia/81f2a

CL: CodeLlama, DS: DeepSeek, CQ: CodeQwen, QC: Qwen2.5-Coder

- **2023:** Exclusively OpenAI models (GPT-3.5 and GPT-4, 7 mentions in total).
- **2024:** GPT remains dominant (42 mentions), while competitors such as Claude (5 mentions) and Gemini (1 mention) begin to appear.
- **2025:** Market expansion with new reasoning models (GPT-o1/o3) and stronger competition from Claude 3.5 and Gemini, although GPT still maintains a clear lead.

(2) Architectural Preferences. Most employed Base LLMs adopt decoder-only architectures, aligning with the mainstream autoregressive generation paradigm. A subset of studies explores encoder–decoder architectures (e.g., CodeT5+ [114]), indicating that architectural diversity remains a promising research direction. In addition, vision–language models (e.g., GPT-4V, LlaVa [65]) are increasingly leveraged to translate visual design specifications or waveform diagrams into Verilog, pointing toward a multimodal future for hardware-oriented LLMs.

4.2 Analysis of Instruction Tuned LLMs

As shown in Table 5, we identify 34 **instruction-tuned** (**IT**) LLMs specifically built for Verilog code generation. A notable characteristic of this ecosystem is its relatively high degree of openness: 19 models (55.9%) release public weights, while 15 (44.1%) remain closed.

4.2.1 Foundation Model. Foundation model choices reveal a clear preference for code-specialized bases. Overall, 28 out of 34 IT LLMs (82.4%) adopt coding-oriented foundations, with three major

clusters: DeepSeek-Coder variants (11 models, 32.4%), Qwen coder series (9 models, 26.5%), and CodeLlama derivatives (8 models, 23.5%). The remaining 6 models (17.6%) build on general-purpose foundations, led by Llama-family models and compact architectures such as GPT-2 and Phi-1.5B, which target more efficient deployment. Furthermore, 7 projects (20.6%) experiment with multiple foundation models, suggesting that systematic comparison across bases is still an active research practice.

4.2.2 Trends and Model Selection Insights. Overall, IT LLM development for Verilog is consolidating around code-specialized foundations (DeepSeek-Coder, Qwen coder series, CodeLlama) and open-weight releases, which together facilitate reproducibility and fair comparison across studies. At the same time, there is a pronounced shift toward explicit reasoning capabilities (e.g., "R1-style" and "Reasoning/Thoughts" variants) to better support long-horizon design and verification tasks.

From a practical perspective, resource-constrained academic environments benefit most from open-source foundations such as DeepSeek-Coder, Llama3.1, and Qwen2.5-Coder, which balance performance, cost, and customizability, albeit at the expense of local compute requirements. For performance-critical applications, state-of-the-art closed-source models (notably GPT and Claude variants) typically deliver the strongest generation and reasoning quality, but incur higher monetary and API-dependency costs. For domain-specific Verilog research, we recommend prioritizing the 19 open-weight IT LLMs as starting points, with particular attention to reasoning-enhanced variants (e.g., CodeV-R1 [142], ReasoningV [91], VeriReason [115], VeriThoughts [131]), which are better aligned with the complex logical inference demands of modern hardware design.

Answer to RQ1: We classify LLMs into two types: Base LLMs and Instruction Tuned (IT) LLMs, where base LLMs include both open-source models and closed-source models. Additionally, 34 domain-specific IT LLMs have been developed, with 19 providing open weights and the majority built upon code-specialized foundation models like DeepSeek-Coder and Qwen2.5-Coder.

5 RQ2: Dataset and Evaluation Metrics for Verilog Code Generation

5.1 Dataset Construction

We categorize datasets for Verilog code generation into two complementary families: (1) *Benchmark datasets* for standardized evaluation, and (2) *Instruct-tuning datasets* for supervised fine-tuning and reasoning augmentation.

5.1.1 Overview. Tables 6 and 7 provide a comprehensive overview of the Verilog code generation dataset landscape. In terms of benchmarking, our analysis reveals 19 open-source benchmarks compared to 9 closed-source alternatives (2023–2025). Similarly, in the instruction-tuning domain, open datasets (22 entries) also outnumber their closed-source items (12 entries).

5.1.2 Benchmark Dataset Construction.

(1) Data Sources. Benchmark datasets for Verilog code generation can be systematically categorized into four primary data sources according to their construction methodologies: template synthesis, mining software repositories, expert curation, and hybrid methods.

① **Template Synthesis.** Template synthesis employs structured templates with parameterized natural-language descriptions and corresponding Verilog code. For example, *DAVE* [84] constructs paired English–Verilog templates (e.g., mapping English "OR" to Verilog "|") and instantiates them via systematic parameter conversion. This process offers strong quality control but inevitably lacks the natural diversity of real-world implementations [31].

Table 6. Benchmark Datasets for Verilog Code Generation (Classified by Availability)

Dataset	Year	Input	Samples	Testbench	Available	URL
Closed-Source						
DAVE_test [84]	2020	Text	250	No	Closed	-
LLM-aided [50]	2024	Text	10	Yes	Closed	-
MCTS [34]	2024	Text	15	Yes	Closed	-
NL2Verilog [107]	2024	Text	8	Yes	Closed	-
VGV [121]	2024	Text+Image	20	Yes	Closed	-
AutoSilicon_test [61]	2025	Text	9	Yes	Closed	-
GEMMV_test [132]	2025	Text	10	Yes	Closed	-
HiVeGen_test [97]	2025	Text	4	Yes	Closed	-
ModelEval_test [130]	2025	Text	94	Yes	Closed	-
Open-Source						
ChipGPT [26]	2023	Text	8	Yes	Open	https://zenodo.org/records/7953725
VeriGen_test [100]	2023	Text	17	Yes	Open	https://github.com/shailja-thakur/VGen
VerilogEval-v1 [66]	2023	Text	156/143	Yes	Open	https://github.com/NVlabs/verilog-eval/tree/release/1.0.0
AutoChip [102]	2024	Text	138	Yes	Open	https://zenodo.org/records/10160723
ChipGPTV [24]	2024	Text+Image	30	Yes	Open	https://github.com/aichipdesign/chipgptv
CreativEval [36]	2024	Text	120	Yes	Open	https://github.com/matthewdelorenzo/CreativEval
Evaluatie_LLMs [21]	2024	Text	8	Yes	Open	https://zenodo.org/records/10947127
Hierarchical [81]	2024	Text	10	Yes	Open	https://github.com/ajn313/ROME-LLM
PyHDL-Eval [19]	2024	Text	168	Yes	Open	https://github.com/cornell-brg/pyhdl-eval
RTLLM-v1 [74]	2024	Text	30	Yes	Open	https://github.com/hkust-zhiyao/RTLLM/tree/v1.1
RTLLM-v2 [70]	2024	Text	50	Yes	Open	https://github.com/hkust-zhiyao/RTLLM/tree/main
RTLRepo_test [17]	2024	Text+Code	1.17k	No	Open	https://huggingface.co/datasets/ahmedallam/RTL-Repo
ArchXBench [90]	2025	Text	51	Yes	Open	https://github.com/sureshpurini/ArchXBench
CVDP [88]	2025	Text	783	Yes	Open	https://github.com/NVlabs/cvdp_benchmark
GenBen [108]	2025	Text+Image	324	Yes	Open	https://github.com/ChatDesignVerification/GenBen
RealBench [58]	2025	Text+Image	60	Yes	Open	https://github.com/IPRC-DIP/RealBench
ResBench [46]	2025	Text	56	Yes	Open	https://github.com/jultrishyyy/ResBench
VerilogEval-v2 [87]	2025	Text	156	Yes	Open	https://github.com/NVlabs/verilog-eval/tree/main/
VeriThoughts [131]	2025	Text	291	No	Open	https://huggingface.co/datasets/wilyub/VeriThoughtsBenchmark

Table 7. Instruct-Tuning Datasets for Verilog Code Generation (Classified by Availability)

Dataset	Year	Input	Samples	Testbench	Available	URL
Closed-Source						
DAVE_train [84]	2020	Text	5k	No	Closed	-
VerilogEval_train [66]	2023	Text	8.5k	No	Closed	-
BetterV [85]	2024	Text	Unknown	No	Closed	-
MEV-LLM [80]	2024	Text	31k	No	Closed	-
CodeV [139]	2025	Text	165k	No	Closed	-
CraftRTL [67]	2025	Text	80K	No	Closed	-
DeepRTL [71]	2025	Text	556k	No	Closed	-
DeepRTL2 [72]	2025	Text	433k	No	Closed	-
GEMMV_train [132]	2025	Text	12k	No	Closed	-
OpenRTLSet [110]	2025	Text	98K	No	Closed	-
ScaleRTL [37]	2025	Text	62k	No	Closed	-
VerilogDB [23]	2025	Text	20k	No	Closed	-
Open-Source						
VeriGen_train [100]	2023	Text	109k	No	Open	https://huggingface.co/datasets/shailja/Verilog_GitHub
AutoVCoder-Data [41]	2024	Text	1M	No	Open	https://github.com/sjtu-zhao-lab/AutoVCoder/tree/main/data
ChipGPT-FT-Data [25]	2024	Text	124k	No	Open	https://modelscope.cn/datasets/changkaiyan/chipgptseries
MG-Verilog-Data [135]	2024	Text	11k	No	Open	https://huggingface.co/datasets/GaTech-EIC/MG-Verilog
Origen-Data [32]	2024	Text	222k	No	Open	https://huggingface.co/datasets/henryen/origen_dataset_instruction
RTLCoder-Data [68, 69]	2024	Text	27K	No	Open	https://github.com/hkust-zhiyao/RTL-Coder/tree/main/dataset
RTLRepo_train [17]	2024	Text+Code	2.92k	No	Open	https://huggingface.co/datasets/ahmedallam/RTL-Repo
Mistral-Verilog [42]	2024	Text	68k	No	Open	https://huggingface.co/datasets/emilgoh/verilog-dataset-v2
CodeV-R1-Data [142]	2025	Text	3k	No	Open	https://huggingface.co/datasets/zhuyaoyu/CodeV-R1-dataset
DeepCircuitX [64]	2025	Text	28k	No	Open	https://drive.google.com/file/d/1Y002eJQPMbrEX7IpmzXDpFRlGl0XgUFu
Haven-KL [129]	2025	Text	62k	No	Open	https://huggingface.co/datasets/yangyiyao/HaVen-KL-Dataset
OpenCores [113]	2025	Text	834	No	Open	https://huggingface.co/datasets/LLM-EDA/opencores
PyraNet [78]	2025	Text	692k	No	Open	https://huggingface.co/datasets/bnadimi/PyraNet-Verilog
ReasoningV-Data [91]	2025	Text	5k	No	Open	https://huggingface.co/datasets/GipAI/ReaoningV
RTL++-Data [15]	2025	Text+Graph	11.7k	No	Open	https://huggingface.co/datasets/makyash/RTL-PP
VeriCoder-Origen [118]	2025	Text	126k	Yes	Open	https://huggingface.co/datasets/LLM4Code/expanded_origen_126k
VeriCoder-RTLCoder [118]	2025	Text	12k	Yes	Open	https://huggingface.co/datasets/LLM4Code/expanded_rtlcoder_12k
VeriLogos-Data [77]	2025	Text	10k	No	Open	https://huggingface.co/datasets/97kjmin/VeriLogos_Augmented_Dataset
VeriPrefer-Data [112]	2025	Text	90k	No	Open	https://huggingface.co/datasets/LLM-EDA/pyra
VeriReason-Data [115]	2025	Text	1k	Yes	Open	https://huggingface.co/Nellyw888/datasets
VeriThoughts-Data [131]	2025	Text	20k	No	Open	https://huggingface.co/datasets/wilyub/VeriThoughtsTrainSet

② **Mining Software Repositories.** This approach extracts problems and code from educational platforms and public repositories, leveraging verified solutions and broad coverage. *VerilogEval* [66] is fully derived from HDLBits [5]: *VerilogEval-machine* uses GPT-3.5-turbo to synthesize 143 problems, while *VerilogEval-human* provides 156 expert-crafted problems. *VerilogEval-v2* [87] extends this with hybrid collections (HDLBits + manual curation), targeting code completion and specification-to-RTL tasks with optimized formats and in-context examples. *AutoChip* [102] also refines HDLBits into 120 executable code-generation tasks from 178 original problems and reconstructs testbenches by reverse-engineering HDLBits' testing logic. In contrast, *RTL-repo-test* [17] mines GitHub repositories created after October 2023 with 4–24 Verilog files under permissive licenses, systematically sampling files and target lines for prediction tasks.

③ **Expert Curated.** Expert-curated datasets are authored by domain specialists based on canonical designs, teaching materials, and industrial practices, typically with precise prompts and comprehensive testbenches. Educationally oriented benchmarks such as *VGV* [121] and *VeriGen_test* [100] draw on course materials and HDLBits-style exercises, covering combinational logic, sequential circuits, and finite state machines, with expert-authored diagrams and testbenches (VGV further includes visual prompts). Standardized, broad-coverage suites like *RTLLM-v1/v2* [70, 74] and *ChipGPTV* [24] define taxonomies across arithmetic, logic/control, storage, and advanced modules. More complex hierarchical and application-level challenges are captured by *ModelEval* [130], *CreativEval* [36], *Hierarchical* [81], and *ArchXBench* [90], which emphasize top–submodule structures, dependency graphs, timing constraints, and diverse domains. *PyHDL-Eval* [19] further advances taxonomy-driven benchmarking with an ontological organization of 168 specification-to-RTL problems across 19 RTL design subcategories, each shipped with Verilog reference solutions and testbenches, enabling finer-grained diagnosis of LLM capabilities across design categories. Application-oriented benchmarks such as *GEMMV* [132] and *CVDP* [88] focus on AI accelerators and design/verification scenarios, respectively, while *ResBench* [46] targets real FPGA applications (e.g., ML acceleration, finance, encryption) with an emphasis on resource optimization. *HiVeGen* [97] further supports accelerator evaluation via LLM-based kernel analysis and dataflow graph extraction.

④ **Hybrid Methods.** Hybrid methods combine open-source mining with expert refinement or LLM-based synthesis to improve both coverage and data quality. *AutoSilicon* [61] mines open-source repositories for industrial modules (I/O controllers, processors, accelerators) and augments them with expert-written prompts aligned with RTLLM description formats. *GenBen* [108] integrates verification projects, textbooks, Stack Overflow discussions, and open-source hardware resources, organizing them into knowledge, design, debugging, and multimodal categories, and applies data perturbation to avoid contamination and improve robustness. *RealBench* [58] builds on four verified open-source IPs (e.g., AES cores, SD controllers, Hummingbirdv2 E203 CPU), providing expert-optimized specifications, 100% line-coverage testbenches, and formal verification for both module- and system-level tasks. *VeriThoughts* [131] combines the MetRex dataset [12] (25.8K synthesizable Verilog designs) with LLM-generated prompts, reasoning traces, and candidate Verilog code (using models such as Gemini and DeepSeek-R1), followed by formal checking and human sampling to ensure correctness.

(2) Quality Assurance. Quality control for benchmark datasets centers on three complementary approaches that ensure reliability and correctness.

① **Expert Curation and Standardization.** Expert curation and standardization reduce ambiguity and enhance reproducibility. Benchmarks adopt expert-authored specifications and reviews, along with unified artifacts that enforce consistent interfaces and evaluation through standardized prompt/module-header formats or structured, annotated diagrams[24, 34, 58, 70, 74, 88, 100, 102, 121].

② **Testbench and Formal/Toolchain Checks.** Testbench and formal/toolchain checks verify functional correctness and implementation viability. Existing work ranges from reverse-engineered testbenches where platform logic is undisclosed [102] to coverage-driven or 100% line-coverage suites [58, 108], hierarchical unit–integration testing [81], and multi-scenario/boundary testing with explicit timing constraints [130]. Formal methods complement simulation via equivalence checking and SAT-based proofs with self-consistency verification [58, 131], while EDA pipelines (e.g., Verilator, Yosys, Design Compiler) provide linting, synthesis, and PPA analysis [26, 97].

③ **Contamination Control.** Contamination control preserves evaluation fairness. Typical strategies include decontamination against external corpora, static/dynamic perturbations with expert verification, multi-stage quality checks assisted by LLM-judge filters with detailed logs to guard against data leakage and noise [58, 70, 88, 108].

5.1.3 Instruct-Tuning Dataset Construction.

(1) Data Sources. Instruct-tuning datasets for Verilog code generation can be systematically categorized into three primary data-source strategies: mining software repositories, LLM synthesis, and hybrid methods.

① **Mining Software Repositories.** Similar to benchmark datasets, this strategy exclusively collects data from public repositories, educational platforms, and community resources without relying on LLM-generated augmentation. *VeriGen_train* [100] integrates GitHub mining with educational resources, processing 70 Verilog textbooks via OCR and text extraction to build a 400MB training corpus alongside GitHub-derived modules. *VerilogDB* [23] and *RTL-repo* [17] extend this approach by incorporating GitHub, OpenCores [8], and academic sources, organizing data into structured databases with rich metadata. This methodology guarantees authenticity and legal compliance but requires heavy preprocessing to handle quality variation and lacks the semantic consistency of LLM-generated descriptions.

② **LLM Synthesis.** LLM synthesis generates training data entirely via language models using instruction-following and structured synthetic data generation. *RTLCoder* [68, 69] exemplifies this with a curated pool of 350 hardware design keywords across more than 10 circuit categories, using GPT-3.5 to create over 27,000 instruction–code pairs. *CraftRTL* [67] adopts Self-Instruct, OSS-Instruct, and Docu-Instruct pipelines to synthesize 80.1k samples, addressing non-textual representations through careful prompt design. *ScaleRTL* [37] leverages DeepSeek-R1 to produce 3.5B-token Chain-of-Thought datasets via automated specification generation and multi-step reasoning synthesis. This strategy offers excellent scalability and consistency, but may under-represent the diversity and practical quirks of human-authored RTL.

③ **Hybrid Methods.** Hybrid methods start from community-sourced code and apply LLM-based enhancement for description generation, quality improvement, or augmentation. *VerilogEval-train* [66] mines GitHub Verilog modules and uses GPT-3.5-turbo to generate 8,502 instruction–code pairs with systematic validation; *BetterV* [85] combines authentic GitHub data with LLM-synthesized "virtual" data, using V2C translation tools and EDA verification to guarantee functional correctness; *MEV-LLM* [80] merges GitHub mining with ChatGPT-3.5-Turbo descriptions to create 31,104 samples labeled across four complexity levels. Multi-source integration is further illustrated by *AutoVCoder* [41], which collects 100,000 RTL modules from GitHub and employs ChatGPT-3.5 for problem–code pair generation with filtering and verification via Icarus Verilog and Python equivalence tests; *MG-Verilog* [135] augments mined repositories with multi-granularity descriptions (from line-level comments to full summaries) using LLaMA2-70B-Chat and GPT-3.5; and *Origen* [32] performs code-to-code augmentation on open-source RTL, with Claude3-Haiku generating descriptions and regenerated code under iterative compilation checks. Expert knowledge integration appears in *Haven* [129], which combines GitHub code with textbook examples and

professional annotations, and in *DeepRTL* [71], which fuses GitHub and proprietary industrial IPs with GPT-4 Chain-of-Thought annotation and professional verification, achieving 90% annotation precision under human evaluation.

(2) Quality Assurance. Quality control for instruct-tuning datasets converges on five complementary approaches that ensure training data reliability and optimize model performance.

① **Syntax and Synthesizability Verification.** This step checks that training samples satisfy basic hardware design requirements via automated toolchains. *VeriGen_train* [100] uses Pyverilog for AST extraction and syntax validation, filtering modules to guarantee complete `module–endmodule` structures and proper port definitions. *AutoVCoder* [41] performs Icarus Verilog compilation followed by Yosys synthesis to ensure mappability to gate-level netlists, removing non-synthesizable constructs (e.g., `initial` blocks, `$display`). *VeriCoder* [118] combines syntax validation with executable testbenches using iverilog to ensure both syntactic and basic behavioral correctness, while *DeepRTL* [71] validates tokenizer compatibility (CodeT5+, 2048-token context) and structural integrity via Pyverilog.

② **Functional Correctness Verification.** Functional verification extends beyond syntax to confirm that code produces expected outputs. *VeriCoder* [118] uses GPT-4o to generate unit-test style testbenches (input stimuli and assertions) and verifies equivalence between generated and reference implementations with iverilog. *VeriThoughts* [131] conducts formal equivalence checking with Yosys `miter` and SAT solvers, guaranteeing identical outputs across all inputs. *ReasoningV* [91] augments simulation with boundary-case analysis (edge cases, reset, error conditions), and *VeriPrefer* [112] applies coverage-driven testing via VCS, achieving over 90% line coverage and assertion validation.

③ **Contamination Control.** Contamination control preserves dataset integrity by removing redundancy and preventing leakage from evaluation benchmarks. *PyraNet* [78] uses Jaccard similarity on tokenized code, discarding samples above 0.9 similarity to avoid overfitting to near-duplicates. *CodeV* [139] adopts MinHash-based deduplication with 128-dimensional vectors to filter duplicates while maintaining diversity. *VeriLogos* [77] compares training data against public benchmarks (VerilogEval, RTLLM) using Rouge-L, removing samples with similarity over 0.5, and *ScaleRTL* [37] uses 5-gram sequence analysis against benchmark solutions to detect and eliminate contamination.

④ **Format Standardization and Length Control.** Standardization ensures consistent representation and compatibility with model constraints. *Haven* [129] normalizes naming conventions (e.g., `clk`, `rst_n`), module declarations, and indentation (4-space), while pruning redundant comments and keeping essential documentation. *OpenRTLSet* [110] uses the CodeLlama tokenizer to enforce an 8k-token limit, and *VeriReason* [115] constrains specification descriptions (100–300 words), reasoning (150–300 words), and code (<2048 tokens) for consistent input formatting. *RTL++* [15] extends standardization to multimodal data, enforcing structured instruction–code–graph triplets and consistent CFG/DFG representations within context limits.

⑤ **Quality Filtering and Human Validation.** Automated filtering is complemented by expert review to ensure annotation accuracy and domain correctness. *PyraNet* [78] employs GPT-4o-mini for 0–20 quality scoring (style, efficiency, standardization), followed by hardware-engineer validation. *DeepRTL* [71] conducts multi-stage expert review with professional Verilog designers and additional engineers, achieving 90% annotation accuracy. *VeriCoder* [118] combines automatic checks with engineer validation of specification–code consistency and testbench effectiveness, reporting 92% functional correctness, while *Haven* [129] leverages textbook examples and professional annotations to build knowledge- and logic-enhanced datasets aligned with engineering practice.

These mechanisms collectively provide reliable, diverse, and functionally correct supervision for instruct-tuning, while mitigating data leakage, annotation errors, and functional inconsistencies.

5.1.4 Trends Analysis. We highlight three key evolutionary aspects of datasets and their implications for evaluation: (1) testbench availability as a proxy for executability, (2) input modality as benchmarks move beyond text-only specifications, and (3) dataset scale as tasks range from reasoning-focused micro-suites to broad coverage collections.

(1) Testbench availability. Executable verification has effectively become the standard for evaluation. Among open benchmarks, 17 of 19 provide testbenches (exceptions: *RTLRepo_test*, which targets line-level generation, and *VeriThoughts*, which uses Yosys-based formal equivalence instead of simulation). In the closed-source category, all datasets except *DAVE_test* include testbenches. By contrast, most instruct-tuning datasets lack executable supervision; only a few (e.g., *VeriCoder-Origen*, *VeriCoder-RTLCoder*, *VeriReason-Data*) incorporate testbenches or similar signals. This gap underscores the need to introduce executable supervision (testbenches, formal specifications, synthesis feedback) earlier in training pipelines.

(2) Input modality. While input specifications remain predominantly text-based, benchmarks are increasingly multimodal (Text+Image), as seen in *ChipGPTV* [24], *VGV* [121], *GenBen* [108], and *RealBench* [58], reflecting realistic workflows where timing diagrams, schematics, and waveforms complement textual requirements. In training resources, instruction corpora are still mostly text-only, with emerging structural representations (e.g., Text+Graph in *RTL++-Data*) that expose hierarchy and connectivity crucial for RTL reasoning.

(3) Dataset scale. Dataset scale diverges with purpose. Benchmarks range from compact reasoning-oriented suites (4–30 items) to mid/large-scale resources (e.g., *CVDP*: 783; *GenBen*: 324) emphasizing coverage and diversity. Instruction-tuning datasets span from specialized compact sets (e.g., *CodeV-R1-Data*) to hundred-thousand or million-scale corpora (e.g., *AutoVCoder-Data*, *PyraNet*), where the former sharpen reasoning and alignment and the latter support broad generalization despite weaker executable validation. This complementarity between small, high-signal benchmarks and large, weakly validated training sets mirrors broader practice in code-oriented LLMs.

Based on these observations, future dataset development should prioritize: (1) early integration of *executable supervision* to narrow the gap between training and evaluation; (2) stronger *multimodal alignment* between training and benchmark specifications; and (3) *reasoning-enhanced training* that explicitly encodes intermediate reasoning and design decisions, as explored by CodeV-R1 [142], VeriReason [115], ScaleRTL [37], and VeriThoughts [131].

5.2 Evaluation Metrics

Assessing the quality of LLM-generated Verilog code is challenging, as it must account for both syntactic validity and semantic correctness. Existing work can be broadly grouped into three categories: similarity-based, execution-based, and LLM-based metrics.

5.2.1 Similarity-based Metrics. Similarity-based metrics evaluate generated Verilog by comparing it to reference implementations without executing the code.

(1) General-Purpose Metrics. Most studies adopt text/code similarity metrics from NLP and general code evaluation:

- **Exact Match:** Binary check of whether generated and reference code are identical [17, 123]
- **Edit Distance Metrics:** Character-level alignment via Levenshtein distance [17, 123] and Ratcliff-Obershelp similarity [84]
- **N-gram Based Metrics:** Sequence-overlap measures such as BLEU [33, 64, 72, 88], METEOR [64, 72], ROUGE [33, 64, 72], and chrF [33], originating from machine translation

These metrics are computationally efficient and easy to implement, but only approximate functional correctness.

(2) Verilog-Specific Metrics. To better capture hardware-specific semantics, SimEval [16] introduces a Verilog-oriented metric operating on three levels: (1) *Syntactic* analysis via PyVerilog ASTs; (2) *Semantic* analysis using CFGs extracted by Verilator; and (3) *Functional* analysis through gate-level netlist comparison after Yosys synthesis. This multi-view design targets RTL-specific characteristics such as hierarchy, concurrency, and register-transfer behavior.

5.2.2 Execution-based Metrics. Execution-based metrics evaluate generated code through compilation and runtime checks, directly measuring functional correctness and implementation quality.

(1) Correctness Assessment. Core execution-based metrics include:

— **Syntax-pass@k:** Fraction of samples that compile without syntax errors within k attempts
— **Functional-pass@k:** Fraction of samples that pass standardized testbenches with expected outputs
— **Formal Verification:** Equivalence checking between generated and reference code using tools such as `Yosys -equiv`

(2) Hardware Quality Metrics. Beyond correctness, some work evaluates hardware implementation quality via PPA (Power, Performance, Area) analysis:

— **Power:** Energy efficiency of synthesized designs
— **Performance:** Timing metrics such as maximum frequency, critical path delay, and latency
— **Area:** Resource utilization in terms of logic, storage elements, and overall silicon footprint

5.2.3 LLM-based Metrics. LLM-based metrics leverage language models themselves to capture deeper semantics and domain knowledge.

(1) Model Confidence Metrics. These metrics analyze internal confidence during generation:

— **Perplexity:** Measures model certainty over generated Verilog sequences [33, 40]

(2) LLM-as-a-Judge. Here, LLMs act as expert evaluators on multiple quality dimensions:

— **GPT Score:** Uses GPT-family models to rate code on readability, correctness, and design best practices [71, 72]
— **VCD-RNK:** Assesses semantic consistency between natural-language specifications and implementations, with knowledge distillation to obtain lightweight evaluators [127]
— **MetRex:** Employs LLMs as PPA predictors for Verilog designs [12]

Answer to RQ2: (1) Verilog datasets fall into two main categories: benchmarks and instruct-tuning corpora. They are constructed via template synthesis, open-source mining, expert curation, LLM-based synthesis, and hybrid pipelines, with quality safeguarded by executable testbenches, formal verification, contamination control, and standardized formats. (2) Evaluation metrics have evolved from text similarity measures toward execution-based assessment, with functional-pass@k becoming the dominant indicator of correctness. Future work should further integrate hardware quality metrics (PPA) and develop LLM-as-a-Judge frameworks for reference-free, semantically faithful evaluation.

6 RQ3: Adaptation and Optimization Techniques for Verilog Code Generation

Building on the datasets and evaluation methodologies discussed in the previous sections, we now examine how LLMs are adapted and optimized for Verilog code generation. These techniques can

Table 8. Comparison of Single-agent EDA-feedback Systems

System	Feedback Source	Refinement Strategy	Optimization Target
VeriPPA [103]	iverilog + Synopsys DC	Dual-stage pipeline	Correctness + PPA
RTL Assistant [56]	Icarus Verilog	Role-based prompts (3 types)	Correctness
RTLFixer [105]	Compiler + RAG	ReAct think-act-observe loops	Syntax repair
Paradigm-Based [96]	PyEDA + Simulator	Paradigm block decomposition	Correctness
EvoVerilog [49]	Testbench + Synthesis	Evolutionary search + non-dominated sorting	Multi-objective
VGV [121]	Simulator	Multimodal visual prompting	Correctness

Table 9. Comparison of Multi-agent EDA-feedback Systems for Verilog Generation

System	#Agents	Architecture Type	EDA Tools	Primary Focus
AIVRIL [106]	2	Generator-Reviewer	Compiler logs	Syntax + functional correctness
LLM-aided Front-End [50]	3	Sequential pipeline	Behavioral simulator	RTL + testbench + review
RTL Agent [92]	2	Generator-Reviewer (Reflexion)	Testbench	Parallel exploration vs. depth
MAGE [140]	4	Role-specialized team	Simulator	Scoring + iterative debug
VerilogCoder [52]	3+	Task planner + verifier + debugger	iverilog + Pyverilog AST	Waveform-level mismatch localization
CoopetitiveV [76]	4	Competitive-cooperative	iverilog + AutoBench	Adversarial refinement
Nexus [95]	Hierarchical	Layered supervisor	Xilinx Vivado	Timing + resource + power
RTLSquad [109]	3 squads	Squad-based (explore/implement/verify)	EDA reports	Interpretable PPA optimization
VFlow [119]	Population	Evolutionary MCTS	Multiple operators	Automated workflow discovery
VeriOpt [98]	4	Planner-Programmer-Reviewer-Evaluator	Synopsys DC	PPA-aware two-stage iteration
AutoSilicon [61]	3	Designer + Debugger + Scheduler	iverilog + Synopsys DC	Industrial-scale RTL
VeriMind [79]	5	Full-pipeline collaboration	EDA integration	pass@ARC metric

be broadly divided into training-free and training-based approaches, each with distinct tradeoffs between computational efficiency and performance gains.

6.1 Training-free Methods

Training-free methods enhance Verilog generation without additional parameter updates, making them especially attractive when computational resources are limited. They mainly rely on EDA-tool feedback, prompt engineering, and inference-time decoding optimization.

6.1.1 EDA-Tool Feedback. Industry-standard verification and synthesis tools provide diagnostics (syntax, functional, timing, PPA) that close the loop from generation to refinement. Approaches split into single-agent (Table 8) and multi-agent (Table 9) frameworks.

(1) Single-Agent Systems. Table 8 contrasts six representative systems. They share a staged feedback loop (correctness first, PPA second) but differ in *how* feedback is consumed: role prompts, ReAct repair, paradigm decomposition, evolutionary search, or multimodal input.

Overall, single-agent systems demonstrate that carefully engineered feedback loops, whether staged, iterative, or evolutionary, can substantially improve correctness and PPA without requiring additional model instances.

(2) Multi-Agent Systems. Table 9 reveals a consistent architectural progression: Generator–Reviewer pairs [92, 106] → role-specialized teams with competitive verification [52, 76, 140] → hierarchical supervisors and automated workflow search [95, 119], culminating in industrial PPA-aware pipelines [61, 79, 98]. The recurring design principle is separation of concerns: generation, verification, and optimization are handled by distinct agents rather than a single monolithic loop.

6.1.2 Prompt Engineering. Two training-free strategies stand out. *Structured prompting* decomposes specifications into intermediate representations before Verilog generation [35, 97, 130]. *RAG* injects domain knowledge at inference time [18, 89]; VeriGRAG [137] extends this with GNN-derived structural embeddings aligned to the LLM via soft prompts.

Table 10. Comparison of Supervised Fine-tuning Approaches for Verilog Generation (One Row Per System)

System	Year	Tuning	Key Technique	Dataset (size)
Data-Centric — compete on dataset construction				
DAVE [84]	2020	Full SFT	English–Verilog template synthesis	DAVE (5k)
VeriGen [100]	2023	Full SFT	GitHub + textbook mining	VeriGen (109k)
RTLCoder [68]	2024	Multi-task LoRA	Keyword-driven LLM synthesis	RTLCoder (27k)
AutoVCoder [41]	2024	LoRA	Hybrid GitHub + LLM pairs	AutoVCoder (1M)
MG-Verilog [135]	2024	QLoRA	Multi-granularity descriptions	MG-Verilog (11k)
CraftRTL [67]	2025	—	Self-/OSS-/Docu-Instruct synthesis	CraftRTL (80k)
Strategy-Centric — reshape how a fixed corpus is consumed				
MEV-LLM [80]	2024	Full SFT	Complexity-routed expert ensemble	MEV-LLM (31k)
DeepRTL [71]	2025	Full SFT	3D curriculum (structure/granularity/quality)	DeepRTL (556k)
PyraNet [78]	2025	LoRA	Hierarchical loss weighting (6 quality layers)	PyraNet (692k)
Multi-task — add auxiliary supervision to generation				
BetterV [85]	2024	Multi-task LoRA	Generator–Discriminator	BetterV
OriGen [32]	2024	Multi-task LoRA	Gen-LoRA + Fix-LoRA (gen–repair)	OriGen (222k)
DeepRTL2 [72]	2025	Full SFT	Generation + contrastive loss	DeepRTL2 (433k)
ITERTL [124]	2025	—	Generation + ranking loss	ITERTL
CodeV [139]	2025	Full SFT	Generation + Fill-in-Middle	CodeV (165k)
Knowledge-Enhanced — inject structure or reasoning				
RTL++ [15]	2025	LoRA	CFG/DFG structural injection	RTL++ (11.7k)
VeriThoughts [131]	2025	Full SFT	DeepSeek-R1 reasoning distillation	VeriThoughts (20k)
CodeV-R1 [142]	2025	DAPO	Reasoning distillation + DAPO	CodeV-R1 (3k)
ScaleRTL [37]	2025	—	CoT reasoning synthesis	ScaleRTL (62k)
ReasoningV [91]	2025	LoRA + Full SFT	Adaptive reasoning depth	ReasoningV (5k)

6.1.3 Inference Optimization. Decoding-time reranking improves candidate quality without parameter updates: syntax-aware temperature scaling [14], execution-based self-consistency clustering [141], and distilled semantic reranking [127].

6.2 Training-based Methods

Training-based methods adapt model parameters and generally outperform training-free approaches at higher compute cost.

6.2.1 Pre-training. Two strategies exist: training from scratch (e.g., Hardware Phi-1.5B [40]) and continued pre-training of general LLMs on Verilog corpora (e.g., FreeV [22] via LoRA).

6.2.2 Supervised Fine-tuning. SFT minimizes $\mathcal{L}_{SFT} = -\sum_{(x,y)} \sum_i \log P(y_i|x, y_{<i}; \theta)$ on specification–code pairs using full fine-tuning, LoRA, or QLoRA. Table 10 lists one row per system; the discussion below highlights cross-cutting patterns only.

(1) Data-Centric Tuning. Innovation lies in *what* is trained on: single-source corpora give way to hybrid pipelines with quality control. On well-curated data, LoRA matches full SFT at lower cost—data quality, not the update strategy, is the primary lever.

(2) Strategy-Centric Tuning. Innovation lies in *how* a fixed corpus is consumed: complexity routing, curriculum ordering, or hierarchical loss weighting each extract gains that plain SFT misses.

(3) Multi-task Tuning. Multi-task tuning (the Multi-task block of Table 10) adds auxiliary supervision to the generation objective along two axes. *Architecturally*, dual-task frameworks pair generation with a complementary capability—quality discrimination (BetterV [85]) or

compiler-feedback-driven repair (OriGen [25, 32]). *In the loss space*, auxiliary objectives such as contrastive loss (RTLCoder [68], DeepRTL2 [72]), ranking loss (ITERTL [124]), decoding constraints [125], or Fill-in-Middle [139] are added to the cross-entropy term. The common motivation is mitigating exposure bias and broadening downstream coverage; the evidence suggests these signals are most effective when they encode task structure (e.g., repair or ranking) that plain next-token prediction cannot capture.

(4) Knowledge-enhanced Tuning. External knowledge enters via CFG/DFG injection or, more recently, reasoning-trace distillation. The latter dominates on complex designs and directly enables the self-correction behaviors exploited by RL (Section 6.2.3).

Overall, the field progresses from competing on data (2020–2024) to competing on training strategy and auxiliary supervision, with the 2025 cohort converging on reasoning distillation plus parameter-efficient tuning—the strongest SFT axis to date and the natural springboard for RL.

6.2.3 Reinforcement Learning. While supervised approaches rely on static datasets, reinforcement learning methods enable dynamic optimization through interaction with the environment, offering potential for continuous improvement in code generation quality. **Reinforcement learning** (**RL**) enables dynamic optimization by training a policy π_θ to maximize expected rewards R:

$$J(\theta) = \mathbb{E}_{x\sim p_{data}, y\sim\pi_\theta(\cdot|x)}[R(y, y^*)] \tag{1}$$

where x represents the natural language specification, y denotes the generated Verilog code, y^* is the reference implementation, and $R(y, y^*)$ quantifies code quality through various metrics including functional correctness and structural similarity. Current approaches are categorized by their reward mechanisms:

(1) Structure-based Rewards. To improve syntactic quality, methods like VeriSeek [113] (using PPO) and VeriReason [115] (using GRPO) incorporate **Abstract Syntax Tree** (**AST**) similarity into the reward function. These approaches typically employ LoRA for efficient iterative optimization, penalizing invalid syntax while rewarding structural alignment with reference code.

(2) Tool-based Feedback. Leveraging EDA tools provides objective correctness signals. VeriLogos [77] utilizes formal equivalence checking, while VeriPrefer [112] and CodeV-R1 [142] employ **Direct Preference Optimization** (**DPO/DAPO**) based on testbench results. By marking code that passes more tests as preferred (y^+), these methods align model outputs with functional verification tools:

$$\mathcal{L}_{DPO} = -\mathbb{E}[\log\sigma(\beta\log\frac{\pi_\theta(y^+|x)}{\pi_{ref}(y^+|x)} - \beta\log\frac{\pi_\theta(y^-|x)}{\pi_{ref}(y^-|x)})]. \tag{2}$$

(3) Multi-objective Optimization. Beyond correctness, frameworks like MCTS-augmented generation [34] model RTL design as a Markov Decision Process. They integrate **Power, Performance, and Area** (**PPA**) metrics into the reward, enabling lookahead-guided decoding that optimizes for downstream hardware constraints.

Empirical evidence demonstrates that RL-based strategies consistently outperform supervised baselines by directly optimizing for the final design goals, whether syntax, functional correctness, or hardware efficiency.

Answer to RQ3: (1) Training-free methods include EDA tool feedback systems (encompassing single-agent iterative refinement and multi-agent collaborative architectures), prompt engineering strategies (hierarchical prompting and RAG-enhanced approaches), and inference optimization techniques (syntax-aware decoding and self-consistency reranking). (2) Training-based methods span pre-training (from-scratch and continued pre-training), supervised fine-tuning (data-centric, strategy-centric, multi-task, and knowledge-enhanced approaches), and reinforcement learning (structure-based rewards, tool-based feedback, and multi-objective optimization).

7 RQ4: Alignment Approaches for Verilog Code Generation

While RQ3 highlighted the capabilities of LLMs in Verilog generation, ensuring these models align with human intentions and rigorous hardware requirements remains critical. Unlike software, hardware design faces strict physical constraints, safety imperatives, and commercial IP risks. Consequently, alignment in this domain encompasses four essential dimensions:

- **Security:** Guaranteeing generated code is free from vulnerabilities and hardware trojans that could compromise system integrity.
- **Efficiency:** Optimizing for PPA to prevent resource wastage and ensure operational viability.
- **Copyright:** Mitigating IP infringement risks by ensuring generated designs do not violate existing patents or licenses.
- **Hallucinations:** Minimizing plausible but functionally incorrect outputs that manifest as logical inconsistencies during implementation.

The following sections examine current methodologies and challenges in addressing these alignment principles.

7.1 Security

Security in LLM-generated Verilog faces two main threats: unintentional vulnerabilities from limited domain knowledge, and intentional backdoors from data poisoning.

(1) Current Methods. Research addresses these via mitigation and defense strategies:

- **Vulnerability Mitigation:** To prevent unintentional flaws, frameworks augment LLMs with security knowledge graphs. SecFSM [55] targets Finite State Machines by retrieving remediation strategies for specific vulnerabilities (e.g., Dead States). Similarly, SecV [38] constructs a Hardware-CWE knowledge graph to enable targeted retrieval and verification of security constraints during generation.
- **Threat Defense:** Addressing external threats involves detecting attacks and cleansing models. RTL-Breaker [75] exposes how training data poisoning can introduce backdoors triggered by specific patterns. Conversely, VeriContaminated [116] detects benchmark contamination in training sets to ensure evaluation integrity. For remediation, SALAD [117] applies machine unlearning to remove sensitive information or malicious patterns while preserving core model utility.

(2) Challenges and Future Directions. Current frameworks remain inadequate for sophisticated hardware threats.

- **Limitations:** ① Existing knowledge integration (e.g., CWE) often lacks depth for circuit-level vulnerabilities; ② defense research lags behind attack methodologies; and ③ risks like jailbreaking or adversarial attacks remain underexplored in the hardware domain.

— **Solutions:** ①**Holistic Evaluation:** Developing frameworks beyond CWE to cover hardware-specific patterns. ②**Multi-stage Verification:** Combining static analysis, formal verification, and simulation to catch subtle flaws. ③ **Adversarial Hardening:** Fine-tuning with adversarial examples to improve robustness. ④ **Hardware Red-Teaming:** Collaborations between security and AI experts to systematically uncover vulnerabilities.

7.2 Efficiency

Optimizing PPA is crucial for hardware viability. Unlike software, physical constraints necessitate rigorous optimization strategies.

(1) Current Methods. Research aligns LLM outputs with PPA goals through three main strategies:

— **Design Space Exploration:** Systematic search identifies optimal implementations. Methods range from MCTS-guided generation (Delorenzo et al. [34]) that rewards PPA efficiency, to ChipGPT's [26] enumerative selection from multiple candidates.
— **Knowledge-Enhanced Optimization:** Injecting domain expertise improves results. VeriOpt [98] uses in-context learning with synthesis reports to select optimization techniques, while Chang et al. [27] leverage program analysis and "Chip Discovery Search" to combine design strengths.
— **Design Decomposition:** Breaking down complexity facilitates optimization. Hierarchical approaches like HiVeGen [97] and AutoSilicon [61] target submodules, while sequential frameworks (LLM-VeriPPA [103], RTLSquad [109]) separate functional verification from PPA refinement or use specialized agents for distinct objectives.

(2) Challenges and Future Directions. Despite these advances, several challenges persist in aligning LLM-generated Verilog code with efficiency requirements.

— **Limitations:** ① LLMs inherently lack physical design awareness: they cannot reliably predict how RTL transformations affect post-synthesis PPA metrics without external EDA feedback; ② even given such metrics, they struggle to reason about competing PPA tradeoffs (e.g., pipelining improves performance but increases area); and ③ Context-dependent constraints and the high computational cost of search-based methods further complicate alignment.
— **Solutions:** ①**Multi-Objective Modeling:** Explicitly modeling tradeoffs to balance competing constraints. ②**Closed-Loop Feedback:** Integ%%%rating EDA tools directly into generation loops to provide concrete PPA signals. ③**Specialized Datasets:** Curating data that pairs descriptions with PPA-optimized code to teach efficiency patterns. ④**Hybrid Architectures:** Combining decomposition, search, and agent-based strategies to manage complexity and computational overhead.

7.3 Copyright

Copyright challenges in hardware generation are twofold: protecting the ownership of generated IP and preventing the reproduction of proprietary designs.

(1) Current Methods. Research addresses these opposing needs via:

— **Ownership Protection:** RTLMarker [111] embeds watermarks through token and statement-level transformations (e.g., parameter encoding, redundant logic). These signatures survive synthesis, enabling ownership verification at both RTL and gate-level netlists.
— **Infringement Prevention:** FreeV [22] mitigates reproduction risks through rigorous dataset curation. By applying multi-layer filtering (license screening, deduplication), it significantly reduces infringement rates in fine-tuned models while maintaining generation quality.

(2) Challenges and Future Directions. Despite these advances, significant copyright alignment challenges persist in LLM-generated Verilog code.

- **Limitations:** ① Current watermarking techniques primarily focus on ownership protection but struggle to balance robustness against detection transparency; ② Existing copyright infringement detection mechanisms rely heavily on surface-level similarity metrics (e.g., cosine similarity).
- **Solutions:** ①**Watermarking:** Techniques must survive complex synthesis optimizations while remaining transparent. Future hierarchical schemes could preserve attribution across abstraction levels (behavioral to gate-level). ②**Infringement Detection:** Surface-level similarity metrics often fail to catch functionally equivalent but syntactically distinct copies. Hardware-specific benchmarks incorporating functional similarity and provenance tracking mechanisms are needed to distinguish between standard interface reuse and actual IP theft.

7.4 Hallucinations

Hallucinations in LLM-generated Verilog code present a significant alignment challenge, manifesting as code that appears syntactically correct but contains functional errors, logical inconsistencies, or design flaws that only become apparent during testing or implementation. Research addressing these challenges has emerged along several promising directions.

(1) Current Methods. Various approaches have been developed to mitigate hallucinations in LLM-generated Verilog code, which can be categorized into three main strategies.

- **Knowledge-Enhanced Generation:** Augmenting models with domain rules reduces errors. HAVEN [129] employs a three-stage pipeline (interpretation, enhancement, reinforcement) based on a specific error taxonomy. HDLCoRe [89] uses HDL-aware Chain-of-Thought for self-verification, while ChatModel [130] leverages multi-agent collaboration to verify designs at functional block levels.
- **Data-Driven refinement:** improving training data quality addresses root causes. RTL++ [15] enriches data with control/data flow graphs to capture structural dependencies. Others focus on error classification [134] or incorporating explicit reasoning paths into fine-tuning datasets (VeriReason [115]).
- **Decoding and Process Optimization:** Modifying generation dynamics improves reliability. DecoRTL [14] uses syntax-aware temperature adaptation and contrastive decoding to reduce redundancy. Paradigm-based methods [96] utilize expert-inspired templates for different circuit types to minimize error propagation.

(2) Challenges and Future Directions.

- **Limitations:** ① Hardware design demands absolute functional correctness, making even minor logical errors fatal; ② Current detection methods are often computationally expensive or limited in scope (e.g., missing timing or edge cases).
- **Solutions:** ①**Verification:** Future work should integrate comprehensive verification (combining formal methods and simulation) directly into generation pipelines. ②**Hybrid Approaches:** Multi-stage approaches that separate structural generation from functional optimization, along with hybrid strategies combining knowledge injection and decoding constraints, offer promising paths to robustly mitigate hallucinations.

Table 11 spans 23 systems across four dimensions. Security work remains attack-heavy; efficiency methods converge on closed-loop PPA feedback; copyright aligns protection (watermarking) with prevention (dataset curation); hallucination mitigation shifts from taxonomy/decoding fixes toward reasoning- and verification-aware training.

Table 11. Comparison of Alignment Approaches for Verilog Generation (One Row Per System)

System	Year	Key Technique	Alignment Target
Security — vulnerability mitigation and threat defense			
SecFSM [55]	2025	KG-guided FSM generation	Prevent unintentional FSM vulnerabilities
SecV [38]	2025	Hardware-CWE KG retrieval	Verify security constraints at generation
RTL-Breaker [75]	2025	Training-data poisoning study	Assess backdoor attack surface
VeriContaminated [116]	2025	Min-K%/CDD contamination detection	Ensure evaluation integrity
SALAD [117]	2025	Machine unlearning	Remove malicious/sensitive patterns
Efficiency — PPA-aware generation and optimization			
MCTS-RTL [34]	2024	PPA-aware MCTS search	Optimize area-delay product
ChipGPT [26]	2023	Enumerative candidate selection	Reduce area/power vs. naive LLM output
VeriOpt [98]	2025	ICL + multi-role LLM	PPA-aware code refinement
Chip Design Agent [27]	2025	Program analysis + discovery search	Reuse-driven PPA improvement
HiVeGen [97]	2025	Hierarchical submodule decomposition	Accelerator-level PPA
AutoSilicon [61]	2025	Task decomposition + multi-version voting	Industrial-scale PPA
VeriPPA [103]	2025	Two-stage correctness → PPA	Sequential functional then PPA tuning
RTLSquad [109]	2025	Squad-based multi-agent debate	Interpretable PPA tradeoffs
Copyright — ownership protection and infringement prevention			
RTLMarker [111]	2025	RTL/netlist watermarking	Traceable IP ownership
FreeV [22]	2025	License-filtered dataset curation	Reduce reproduction of proprietary RTL
Hallucinations — functional correctness and error mitigation			
HAVEN [129]	2025	Error taxonomy + 3-stage pipeline	Domain-knowledge correction
HDLCoRe [89]	2025	HDL-aware CoT + self-verification	Rule-guided generation without tools
ChatModel [130]	2025	Multi-agent IR + block verification	Hierarchical design correctness
RTL++ [15]	2025	CFG/DFG graph enrichment	Structural dependency modeling
RTL Error Study [134]	2025	Error classification + remediation rules	Targeted error mitigation
VeriReason [115]	2025	Reasoning traces + GRPO	Functional correctness via RL
DecoRTL [14]	2025	Syntax-aware temperature + reranking	Decoding-time reliability
Paradigm-Based [96]	2025	Expert paradigm templates	Type-specific circuit generation

Answer to RQ4: Our analysis reveals that alignment of LLM-generated Verilog code requires addressing four key dimensions: security, efficiency, copyright, and hallucinations. Table 11 lists 23 alignment systems across security, efficiency, copyright, and hallucinations, each targeting a distinct alignment objective. Current approaches demonstrate promising results but still face significant challenges. Future research should focus on developing comprehensive verification frameworks, multi-stage generation pipelines, and hybrid approaches that combine multiple alignment strategies.

8 The Road Ahead

In this section, we first synthesize quantitative evidence across reviewed studies, then analyze current limitations, and finally outline a research roadmap for next-generation Verilog generation.

8.1 Cross-Study Evidence Synthesis

While RQ1–RQ4 provide a taxonomic organization of the field, this subsection offers a structured cross-study comparison to draw evidence-based conclusions. Since substantial heterogeneity in experimental setups (prompt templates, sampling temperatures, benchmark versions, hardware environments) renders direct comparison of absolute numbers across papers unreliable, we adopt two complementary strategies: (1) *relative improvement analysis*, extracting within-paper gains under controlled conditions; and (2) *convergent finding identification*, synthesizing conclusions consistently supported by multiple independent studies.

Table 12. Relative Improvement (Δpass@1) Over Baselines, Extracted from Controlled within-paper Experiments

Category	Method	Base Model	Benchmark	Δpass@1	Baseline
Training-free Methods					
EDA Feedback (Single)	RTLFixer [105]	GPT-4	VerilogEval-v1	+10.1%	Zero-shot GPT-4
EDA Feedback (Multi)	VerilogCoder [52]	GPT-4 Turbo	VerilogEval-v2	+33.9%	Zero-shot GPT-4 Turbo
Prompt Engineering	HDLCoRe [89]	Qwen2.5-72B	RTLLM-v2	+10.0%	Zero-shot Qwen2.5-72B
Inference Optimization	VRank [141]	GPT-4o	VerilogEval-v1	+9.2%	Greedy decoding
Supervised Fine-Tuning					
Data-Centric	RTLCoder [68]	DeepSeek-Coder-6.7B	VerilogEval-v1	+11.4%	Base DeepSeek-Coder-6.7B
Data-Centric	CodeV [139]	DeepSeek-Coder-6.7B	VerilogEval-v1	+22.8%	Base DeepSeek-Coder-6.7B
Strategy-Centric	PyraNet [78]	CodeLlama-7B	VerilogEval-v1	+3.8%	Standard SFT
Knowledge-Enhanced	VeriThoughts [131]	Qwen2.5-14B	VerilogEval-v1	+13.0%	Base Qwen2.5-14B
Reinforcement Learning					
Structure-based	VeriSeek [113]	DeepSeek-Coder-6.7B	VerilogEval-v1	+8.3%	SFT-only
Tool-based	VeriPrefer [112]	DeepSeek-Coder-6.7B	VerilogEval-v1	+3.1%	SFT-only

8.1.1 Relative Improvement Analysis. Table 12 reports within-paper improvements of representative methods over their respective baselines under controlled conditions. Because entries span different benchmarks, cross-row comparisons should be interpreted with caution.

Three patterns emerge. (1) SFT delivers larger gains than prompt engineering or inference optimization among single-model methods, while multi-agent systems (e.g., VerilogCoder) achieve the highest improvements at the expense of increased API costs. (2) RL further improves over SFT baselines by 3–8 pp, suggesting that execution-aware optimization complements supervised learning. (3) Reasoning distillation yields substantial gains on larger backbones (e.g., VeriThoughts +13.0% on 14B), offering a qualitatively distinct advantage over data-centric scaling.

8.1.2 Convergent Findings Across Independent Studies. We further identify findings that converge across multiple studies despite differing experimental setups:

Finding 1: Domain-adapted small models rival general-purpose large models. Multiple studies [41, 68, 71, 139] independently show that 7B models fine-tuned on Verilog data match or exceed GPT-3.5-turbo and GPT-4 on functional correctness, providing strong evidence that domain specialization compensates for scale.

Finding 2: RL over SFT yields consistent but bounded gains. All RL-based studies [77, 112, 113, 115, 142] report 3–8 pp improvements over SFT baselines across different algorithms (PPO, GRPO, DPO), indicating that RL addresses exposure bias and the absence of execution-aware optimization in supervised training.

Finding 3: Multi-agent systems excel on complex designs. Comparisons between single- and multi-agent architectures [52, 61, 76, 140] consistently show gains that scale with design complexity (e.g., VerilogCoder +33.9% on VerilogEval-v2), suggesting that agent specialization with iterative EDA feedback is most valuable when tasks exceed single-model capacity.

Finding 4: Reasoning distillation is a pivotal 2025 innovation. Works leveraging reasoning traces from frontier models (e.g., DeepSeek-R1) [37, 91, 131, 142] consistently report substantial improvements, with VeriThoughts achieving +13.0% on a 14B backbone. Unlike data scaling, reasoning distillation introduces structured problem decomposition, addressing a qualitative bottleneck in hardware design understanding rather than merely increasing exposure to Verilog tokens.

8.1.3 Threats to Comparability. Several factors limit the above conclusions to directional trends rather than precise effect sizes:

— **Benchmark versioning:** VerilogEval-v1 and v2 differ in difficulty and testbench coverage, precluding cross-version comparison.

— **Prompt sensitivity:** Variations in prompts can shift pass@1 by 5–15 pp even on identical benchmarks [87].
— **Evaluation protocol:** Differences in temperature, n in pass@k, and post-processing introduce systematic bias.
— **Unreported details:** Many studies omit confidence intervals or generation parameters, precluding formal meta-analysis.

Nevertheless, the convergent findings, derived from consistent directional conclusions across heterogeneous settings, offer the community reliable evidence-based guidance on the relative merits of competing paradigms.

8.2 Limitations

Based on our analysis across RQ1–RQ4 and the synthesis above, we identify four key limitations of current research:

(1) Foundation and Knowledge Gap. General-purpose LLMs lack intrinsic understanding of hardware constraints (timing, concurrency, area). Predominant autoregressive architectures struggle with Verilog's parallel semantics, often producing code that simulates correctly but fails synthesis. *Future Direction: Develop specialized architectures pre-trained on hardware corpora with integrated EDA feedback.*

(2) Data and Benchmark Scarcity. High-quality Verilog data is scarce compared to software languages, often lacking documentation or testbenches. Existing benchmarks are small-scale (<100 samples), focusing on basic modules while neglecting system-level complexities (e.g., pipelined datapaths, SoCs). *Future Direction: Construct large-scale, uncontaminated benchmarks with graduated complexity and comprehensive verification suites.*

(3) Evaluation and Alignment Deficits. Current metrics (functional-pass@k) ignore critical hardware dimensions like PPA and security. Alignment research is nascent: security tools miss hardware-specific vulnerabilities, PPA optimization lacks efficient feedback loops, and copyright mechanisms are fragile. *Future Direction: Establish standardized frameworks integrating synthesis-based PPA assessment, security analysis, and robust IP protection.*

(4) Deployment Readiness. No current solution integrates seamlessly into industrial EDA workflows. The lack of interactive refinement, explainability, and human-in-the-loop features hinders practical adoption by hardware engineers. *Future Direction: Prioritize IDE-integrated tools supporting iterative collaboration and explainable design decisions.*

8.3 Roadmap

Building on the evidence synthesis and identified limitations, we propose a three-stage trajectory to advance the field from isolated module generation toward integrated, system-level solutions.

8.3.1 Stage 1: Strengthening Foundations.

— **Hardware-Aware Models:** Move beyond general-purpose LLMs by developing billion-scale models pre-trained on hardware-specific corpora. Key innovations should include hardware-aware tokenization and architectural modifications to model concurrency.
— **Comprehensive Benchmarking:** Construct living benchmarks exceeding 1,000 samples, covering the full spectrum from basic gates to industrial SoCs, grounded with synthesis-verified PPA truth.

8.3.2 Stage 2: Expanding Capabilities.

— **System-Level Hierarchical Generation:** Shift focus from single modules to entire systems. This requires automatic decomposition, interface synthesis, and hierarchical planning to manage cross-module dependencies and protocols.

— **Multimodal Inputs:** Enable generation from diverse engineering artifacts (timing diagrams, block schematics, and waveforms), allowing designers to define architectures visually.

8.3.3 Stage 3: Optimization and Deployment.

— **Closed-Loop PPA and Security:** Replace open-loop generation with reinforcement learning pipelines driven by direct EDA feedback (synthesis reports, formal verification), ensuring outputs meet physical and security constraints by construction.
— **Human-AI Collaboration:** Develop "Copilot-for-Hardware" interfaces that support interactive refinement, explain design choices, and learn from user corrections, facilitating integration into production flows.

By synthesizing these directions, the community can transition from experimental prototypes to trustworthy, expert-level assistants that fundamentally transform the electronic design automation ecosystem.

9 Threats to Validity

Paper search omission. Despite searching six databases and employing snowballing to identify 102 relevant papers (including 32 preprints), the fast-paced nature of LLM research may lead to overlooking emerging work. We mitigated this through rigorous keyword selection and multi-stage filtering.
Study selection bias. The reduction from initial candidates to the final set involved subjective quality assessments. To address this, we enforced strict inclusion criteria and a standardized Likert-scale framework, requiring a threshold score (12/15) for all entries.
Categorization and analysis bias. Categorizing diverse methodologies (models, datasets, alignment strategies) introduces potential classification bias. We minimized this by aligning with established taxonomies and focusing on factual extraction over subjective interpretation, though our roadmap inevitably reflects our synthesized perspective on the field's trajectory.

10 Conclusion

This systematic literature review analyzes the transformative impact of Large Language Models on Verilog code generation, synthesizing insights from 102 studies (2020–2025). We comprehensively mapped the landscape of base and instruction-tuned models (RQ1), scrutinized the critical role of curated datasets and evaluation metrics (RQ2), and categorized optimization strategies ranging from prompt engineering to reinforcement learning (RQ3). Furthermore, we highlighted essential alignment dimensions (i.e., security, efficiency, copyright, and hallucinations) critical for practical deployment (RQ4). By identifying key limitations and proposing a multi-stage research roadmap, this work aims at guiding the community in transitioning from experimental prototypes to robust, system-level hardware design assistants, ultimately reshaping the future of Electronic Design Automation.

References

[1] [n. d.]. *ACM Digital Library*. Retrieved November 1, 2025 from https://dl.acm.org/
[2] [n. d.]. *Anthropic's Claude series models*. Retrieved November 1, 2025 from https://docs.claude.com/en/docs/about-claude/models/overview/
[3] [n. d.]. *arXiv Database*. Retrieved November 1, 2025 from https://arxiv.org/
[4] [n. d.]. *Google's Gemini series models*. Retrieved November 1, 2025 from https://ai.google.dev/gemini-api/docs/models/
[5] [n. d.]. *HDLBits Website*. Retrieved November 1, 2025 from https://hdlbits.01xz.net/wiki/Main_Page
[6] [n. d.]. *IEEE Xplore Database*. Retrieved November 1, 2025 from https://ieeexplore.ieee.org/
[7] [n. d.]. *OpenAI's GPT series models*. Retrieved November 1, 2025 from https://platform.openai.com/docs/models/
[8] [n. d.]. *OpenCores Website*. Retrieved November 1, 2025 from https://opencores.org/
[9] [n. d.]. *ScienceDirect Database*. Retrieved November 1, 2025 from https://www.sciencedirect.com/

[10] [n. d.]. *SpringerLink Database*. Retrieved November 1, 2025 from https://link.springer.com/
[11] [n. d.]. *Web of Science Database*. Retrieved November 1, 2025 from https://www.webofscience.com/
[12] Manar Abdelatty, Jingxiao Ma, and Sherief Reda. 2025. MetRex: A benchmark for verilog code metric reasoning using LLMs. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, Yuichi Nakamura and Yu Wang (Eds.). ACM, 995–1001. DOI: https://doi.org/10.1145/3658617.3697625
[13] Meisam Abdollahi, Seyedeh Faegheh Yeganli, Mohammad Baharloo, and Amirali Baniasadi. 2024. Hardware design and verification with large language models: A scoping review, challenges, and open issues. *Electronics* 14, 1 (2024), 120.
[14] Mohammad Akyash, Kimia Zamiri Azar, and Hadi Mardani Kamali. 2025. DecoRTL: A Run-time Decoding Framework for RTL Code Generation with LLMs. arXiv:2507.02226. Retrieved from https://arxiv.org/abs/2507.02226
[15] Mohammad Akyash, Kimia Zamiri Azar, and Hadi Mardani Kamali. 2025. RTL++: Graph-enhanced LLM for RTL Code Generation. arXiv:2505.13479. Retrieved from https://arxiv.org/abs/2505.13479
[16] Mohammad Akyash and Hadi Mardani Kamali. 2025. SimEval: Investigating the similarity obstacle in LLM-based hardware code generation. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, Yuichi Nakamura and Yu Wang (Eds.). ACM, 1002–1007. DOI: https://doi.org/10.1145/3658617.3697624
[17] Ahmed Allam and Mohamed Shalan. 2024. Rtl-repo: A benchmark for evaluating LLMs on large-scale rtl design projects. *2024 IEEE LLM Aided Design Workshop* (2024), 1–5.
[18] Kangbo Bai, Peiran Yan, Lifeng Liu, and Tianyu Jia. 2025. VeriRAG: Design AI-specific CPU co-processor with RAG-Enhanced LLMs. In *Proceedings of the 2025 International Symposium of Electronics Design Automation*. 76–82. DOI: https://doi.org/10.1109/ISEDA65950.2025.11100783
[19] Christopher Batten, Nathaniel Pinckney, Mingjie Liu, Haoxing Ren, and Brucek Khailany. 2024. PyHDL-Eval: An LLM evaluation framework for hardware design using Python-embedded DSLs. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*. 1–17. DOI: https://doi.org/10.1145/3670474.3685948
[20] Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, et. al, 2024. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. arXiv:2401.02954. Retrieved from https://arxiv.org/abs/2401.02954
[21] Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. 2024. Evaluating LLMs for Hardware Design and Test. arXiv:2405.02326. Retrieved from https://arxiv.org/abs/2405.02326
[22] Sam Bush, Matthew DeLorenzo, Phat Tieu, and Jeyavijayan Rajendran. 2025. Free and fair hardware: A pathway to copyright infringement-free verilog generation using LLMs. In *Proceedings of the 62nd ACM/IEEE Design Automation Conference*. IEEE, 1–7. DOI: https://doi.org/10.1109/DAC63849.2025.11132658
[23] Paul E. Calzada, Zahin Ibnat, Tanvir Rahman, Kamal Kandula, Danyu Lu, Sujan Kumar Saha, Farimah Farahmandi, and Mark Tehranipoor. 2025. VerilogDB: The Largest, Highest-Quality Dataset with a Preprocessing Framework for LLM-based RTL Generation. arXiv:2507.13369. Retrieved from https://arxiv.org/abs/2507.13369
[24] Kaiyan Chang, Zhirong Chen, Yunhao Zhou, Wenlong Zhu, Kun Wang, Haobo Xu, Cangyuan Li, Mengdi Wang, Shengwen Liang, Huawei Li, Yinhe Han, and Ying Wang. 2024. Natural language is not enough: Benchmarking multi-modal generative AI for Verilog generation. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, Jinjun Xiong and Robert Wille (Eds.). ACM, 117:1–117:9. DOI: https://doi.org/10.1145/3676536.3676679
[25] Kaiyan Chang, Kun Wang, Nan Yang, Ying Wang, Dantong Jin, Wenlong Zhu, Zhirong Chen, Cangyuan Li, Hao Yan, Yunhao Zhou, Zhuoliang Zhao, Yuan Cheng, Yudong Pan, Yiqi Liu, Mengdi Wang, Shengwen Liang, Yinhe Han, Huawei Li, and Xiaowei Li. 2024. Data is all you need: Finetuning LLMs for Chip Design via an Automated design-data augmentation framework. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*, Vivek De (Ed.). ACM, 60:1–60:6. DOI: https://doi.org/10.1145/3649329.3657356
[26] Kaiyan Chang, Ying Wang, Haimeng Ren, Mengdi Wang, Shengwen Liang, Yinhe Han, Huawei Li, and Xiaowei Li. 2023. ChipGPT: How far are we from natural language hardware design. arXiv:2305.14019. Retrieved from https://arxiv.org/abs/2305.14019
[27] Kaiyan Chang, Wenlong Zhu, Kun Wang, Xinyang He, Nan Yang, Zhirong Chen, Dantong Jin, Cangyuan Li, Yunhao Zhou, Hao Yan, Zhuoliang Zhao, Yuan Cheng, Mengdi Wang, Shengwen Liang, Yinhe Han, Xiaowei Li, Huawei Li, and Ying Wang. 2025. A data-centric chip design agent framework for Verilog code generation. *ACM Transactions on Design Automation of Electronic Systems* 30, 6 (2025), 27 pages. DOI: https://doi.org/10.1145/3727980
[28] Lei Chen, Yiqi Chen, Zhufei Chu, Wenji Fang, Tsung-Yi Ho, Ru Huang, Yu Huang, Sadaf Khan, Min Li, Xingquan Li, et. al, 2024. Large circuit models: Opportunities and challenges. *Science China Information Sciences* 67, 10 (2024). DOI: https://doi.org/10.1007/S11432-024-4155-7
[29] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et. al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374. Retrieved from https://arxiv.org/abs/2107.03374

[30] Xiangping Chen, Xing Hu, Yuan Huang, He Jiang, Weixing Ji, Yanjie Jiang, Yanyan Jiang, Bo Liu, Hui Liu, Xiaochen Li, Xiaoli Lian, Guozhu Meng, Xin Peng, Hailong Sun, Lin Shi, Bo Wang, Chong Wang, Jiayi Wang, Tiantian Wang, Jifeng Xuan, Xin Xia, Yibiao Yang, Yixin Yang, Li Zhang, Yuming Zhou, and Lu Zhang. 2025. Deep learning-based software engineering: Progress, challenges, and opportunities. *Science China Information Sciences* 68, 1 (2025). DOI: https://doi.org/10.1007/S11432-023-4127-5

[31] Xiang Chen, Guang Yang, Z Cui, G. Meng, and Zan Wang. 2021. Survey of state-of-the-art automatic code comment generation ruan jian xue bao. *Journal of Software* 32, 7 (2021), 2118–2141.

[32] Fan Cui, Chenyang Yin, Kexing Zhou, Youwei Xiao, Guangyu Sun, Qiang Xu, Qipeng Guo, Yun Liang, Xingcheng Zhang, Demin Song, and Dahua Lin. 2024. OriGen: Enhancing RTL code generation with code-to-code augmentation and self-reflection. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, Jinjun Xiong and Robert Wille (Eds.). ACM, 99:1–99:9. DOI: https://doi.org/10.1145/3676536.3676830

[33] Enrique Dehaerne, Bappaditya Dey, Sandip Halder, and Stefan De Gendt. 2023. A Deep Learning Framework for Verilog Autocompletion Towards Design and Verification Automation. arXiv:2304.13840. Retrieved from https://arxiv.org/abs/2304.13840

[34] Matthew DeLorenzo, Animesh Basak Chowdhury, Vasudev Gohil, Shailja Thakur, Ramesh Karri, Siddharth Garg, and Jeyavijayan Rajendran. 2024. Make Every Move Count: LLM-based High-Quality RTL Code Generation Using MCTS. arXiv:2402.03289. Retrieved from https://arxiv.org/abs/2402.03289

[35] Matthew DeLorenzo, Kevin Tieu, Prithwish Jana, Piyush Jha, Dileep Kalathil, Vijay Ganesh, and Jeyavijayan Rajendran. 2025. Abstractions-of-Thought: Intermediate Representations for LLM Reasoning in Hardware Design. arXiv:2505.15873. Retrieved from https://arxiv.org/abs/2505.15873

[36] Matthew DeLorenzo, Phat Tieu, Chen Chen, Vasudev Gohil, and Jeyavijayan Rajendran. 2025. Watermarking LLMs - challenges and opportunities in electronic design automation. In *Proceedings of the IEEE International Conference on Omni-layer Intelligent Systems*. IEEE, 1–4. DOI: https://doi.org/10.1109/COINS65080.2025.11125763

[37] Chenhui Deng, Yun-Da Tsai, Guan-Ting Liu, Zhongzhi Yu, and Haoxing Ren. 2025. ScaleRTL: Scaling LLMs with reasoning data and test-time compute for accurate RTL code generation. In *Proceedings of the 7th ACM/IEEE Symposium on Machine Learning for CAD*. IEEE, 1–9. DOI: https://doi.org/10.1109/MLCAD65511.2025.11189212

[38] Fanghao Fan, Yingjie Xia, and Li Kuang. 2025. SecV: LLM-based secure verilog generation with clue-guided exploration on hardware-CWE knowledge graph. In *Proceedings of the 34th International Joint Conference on Artificial Intelligence*. ijcai.org, 8049–8057. DOI: https://doi.org/10.24963/IJCAI.2025/895

[39] Wenji Fang, Jing Wang, Yao Lu, Shang Liu, Yuchao Wu, Yuzhe Ma, and Zhiyao Xie. 2025. A Survey of Circuit Foundation Model: Foundation AI Models for VLSI Circuit Design and EDA. arXiv:2504.03711. Retrieved from https://arxiv.org/abs/2504.03711

[40] Weimin Fu, Shijie Li, Yifang Zhao, Haocheng Ma, Raj Dutta, Xuan Zhang, Kaichen Yang, Yier Jin, and Xiaolong Guo. 2024. Hardware Phi-1.5B: A large language model encodes hardware domain specific knowledge. In *Proceedings of the 2024 29th Asia and South Pacific Design Automation Conference*. 349–354. DOI: https://doi.org/10.1109/ASP-DAC58780.2024.10473927

[41] Mingzhe Gao, Jieru Zhao, Zhe Lin, Wenchao Ding, Xiaofeng Hou, Yu Feng, Chao Li, and Minyi Guo. 2024. AutoV-Coder: A systematic framework for automated verilog code generation using LLMs. In *Proceedings of the 42nd IEEE International Conference on Computer Design*. IEEE, 162–169. DOI: https://doi.org/10.1109/ICCD63220.2024.00033

[42] Emil Goh, Maoyang Xiang, I-Chyn Wey, and Tee Hui Teo. 2024. From English to ASIC: Hardware Implementation with Large Language Model. arXiv:2403.07039. Retrieved from https://arxiv.org/abs/2403.07039

[43] Xungang Gu, Mengqi Wang, Yangjie Tian, Ning Li, Jiaze Sun, Jingfang Xu, He Zhang, Ruohua Xu, and Ming Liu. 2025. A comprehensive approach to instruction tuning for Qwen2.5: Data selection, domain interaction, and training protocols. *Comput.* 14, 7 (2025), 264. DOI: https://doi.org/10.3390/COMPUTERS14070264

[44] Yi Gui, Zhen Li, Yao Wan, Yemin Shi, Hongyu Zhang, Bohua Chen, Yi Su, Dongping Chen, Siyuan Wu, Xing Zhou, Wenbin Jiang, Hai Jin, and Xiangliang Zhang. 2025. WebCode2M: A real-world dataset for code generation from webpage designs. In *Proceedings of the ACM on Web Conference 2025*, Guodong Long, Michale Blumestein, Yi Chang, Liane Lewin-Eytan, Zi Helen Huang, and Elad Yom-Tov (Eds.). ACM, 1834–1845. DOI: https://doi.org/10.1145/3696410.3714889

[45] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Harkirat Singh Behl, Xin Wang, Sébastien Bubeck, Ronen Eldan, Adam Tauman Kalai, Yin Tat Lee, and Yuanzhi Li. 2023. Textbooks Are All You Need. arXiv:2306.11644. Retrieved from https://arxiv.org/abs/2306.11644

[46] Ce Guo and Tong Zhao. 2025. ResBench: A resource-aware benchmark for LLM-generated FPGA designs. In *Proceedings of the 15th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies, HEART 2025, Kumamoto, Japan, May 26-28, 2025*, Tomohiro Ueno, Ameer Abdelhadi, Dirk Koch, Yasunori Osana, and Yukinori Sato (Eds.). ACM, 25–34. DOI: https://doi.org/10.1145/3728179.3728192

[47] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv:2501.12948. Retrieved from https://arxiv.org/abs/2501.12948

[48] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence. arXiv:2401.14196. Retrieved from https://arxiv.org/abs/2401.14196

[49] Ping Guo, Yiting Wang, Wanghao Ye, Yexiao He, Ziyao Wang, Xiaopeng Dai, Ang Li, and Qingfu Zhang. 2025. EvoVerilog: Large Langugage Model Assisted Evolution of Verilog Code. arXiv:2508.13156. Retrieved from https://arxiv.org/abs/2508.13156

[50] Vyom Kumar Gupta, Abhishek Yadav, Masahiro Fujita, and Binod Kumar. 2024. LLM-aided front-end design framework for early development of verified RTLs. In *Proceedings of the 33rd IEEE Asian Test Symposium*. IEEE, 1–6. DOI: https://doi.org/10.1109/ATS64447.2024.10915239

[51] Zhuolun He and Bei Yu. 2024. Large language models for EDA: Future or mirage? In *Proceedings of the 2024 International Symposium on Physical Design*, Iris Hui-Ru Jiang and Gracieli Posser (Eds.). ACM, 65–66. DOI: https://doi.org/10.1145/3626184.3639700

[52] Chia-Tung Ho, Haoxing Ren, and Brucek Khailany. 2025. VerilogCoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (AST)-based waveform tracing tool. In *Proceedings of the AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence*, Toby Walsh, Julie Shah, and Zico Kolter (Eds.). AAAI Press, 300–307. DOI: https://doi.org/10.1609/AAAI.V39I1.32007

[53] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 79 pages. DOI: https://doi.org/10.1145/3695988

[54] Xing Hu, Feifei Niu, Junkai Chen, Xin Zhou, Junwei Zhang, Junda He, Xin Xia, and David Lo. 2025. Assessing and Advancing Benchmarks for Evaluating Large Language Models in Software Engineering Tasks. arXiv:2505.08903. Retrieved from https://arxiv.org/abs/2505.08903

[55] Ziteng Hu, Yingjie Xia, Xiyuan Chen, and Li Kuang. 2025. SecFSM: Knowledge Graph-Guided Verilog Code Generation for Secure Finite State Machines in Systems-on-Chip. arXiv:2508.12910. Retrieved from https://arxiv.org/abs/2508.12910

[56] Hanxian Huang, Zhenghan Lin, Zixuan Wang, Xin Chen, Ke Ding, and Jishen Zhao. 2024. Towards LLM-Powered Verilog RTL Assistant: Self-Verification and Self-Correction. arXiv:2406.00115. Retrieved from https://arxiv.org/abs/2406.00115

[57] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A Survey on Large Language Models for Code Generation. arXiv:2406.00515. Retrieved from https://arxiv.org/abs/2406.00515

[58] Pengwei Jin, Di Huang, Chongxiao Li, Shuyao Cheng, Yang Zhao, Xinyao Zheng, Jiaguo Zhu, Shuyi Xing, Bohan Dou, Rui Zhang, Zidong Du, Qi Guo, and Xing Hu. 2025. RealBench: Benchmarking Verilog Generation Models with Real-World IP Designs. arXiv:2507.16200. Retrieved from https://arxiv.org/abs/2507.16200

[59] Sathvik Joel, Jie Wu, and Fatemeh Fard. 2025. A survey on LLM-based code generation for low-resource and domain-specific programming languages. *ACM Transactions on Software Engineering and Methodology* (2025). DOI: https://doi.org/10.1145/3770084

[60] Barbara A. Kitchenham, Lech Madeyski, and David Budgen. 2023. SEGRESS: Software engineering guidelines for reporting secondary studies. *IEEE Transactions on Software Engineering* 49, 3 (2023), 1273–1298. DOI: https://doi.org/10.1109/TSE.2022.3174092

[61] Cangyuan Li, Chujie Chen, Yudong Pan, Wenjun Xu, Yiqi Liu, Kaiyan Chang, Yujie Wang, Mengdi Wang, Huawei Li, Yinhe Han, et al. 2025. Autosilicon: Scaling up RTL design generation capability of large language models. *ACM Transactions on Design Automation of Electronic Systems* 30, 6 (2025), 1–21.

[62] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! arXiv:2305.06161. Retrieved from https://arxiv.org/abs/2305.06161

[63] Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. 2023. Textbooks Are All You Need II: phi-1.5 technical report. arXiv:2309.05463. Retrieved from https://arxiv.org/abs/2309.05463

[64] Zeju Li, Changran Xu, Zhengyuan Shi, Zedong Peng, Yi Liu, Yunhao Zhou, Lingfeng Zhou, Chengyu Ma, Jianyuan Zhong, Xi Wang, Jieru Zhao, Zhufei Chu, Xiaoyan Yang, and Qiang Xu. 2025. DeepCircuitX: A Comprehensive Repository-Level Dataset for RTL Code Understanding, Generation, and PPA Analysis. arXiv:2502.18297. Retrieved from https://arxiv.org/abs/2502.18297

[65] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual instruction tuning. *Advances in neural information processing systems* 36 (2023), 34892–34916. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2023/hash/6dcf277ea32ce3288914faf369fe6de0-Abstract-Conference.html

[66] Mingjie Liu, Nathaniel Ross Pinckney, Brucek Khailany, and Haoxing Ren. 2023. Invited paper: VerilogEval: Evaluating large language models for verilog code generation. In *Proceedings of the IEEE/ACM International Conference on Computer Aided Design*. IEEE, 1–8. DOI: https://doi.org/10.1109/ICCAD57390.2023.10323812

[67] Mingjie Liu, Yun-Da Tsai, Wenfei Zhou, and Haoxing Ren. 2025. CraftRTL: High-quality synthetic data generation for verilog code models with correct-by-construction non-textual representations and targeted code repair. In *Proceedings of the 13th International Conference on Learning Representations*. OpenReview.net. Retrieved from https://openreview.net/forum?id=8KQzoD5XAr

[68] Shang Liu, Wenji Fang, Yao Lu, Jing Wang, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2025. RTLCoder: Fully open-source and efficient LLM-assisted RTL code generation technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 4 (2025), 1448–1461. DOI: https://doi.org/10.1109/TCAD.2024.3483089

[69] Shang Liu, Wenji Fang, Yao Lu, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2024. RTLCoder: Outperforming GPT-3.5 in design RTL generation with our open-source dataset and lightweight solution. In *Proceedings of the 2024 IEEE LLM Aided Design Workshop*. 1–5. DOI: https://doi.org/10.1109/LAD62341.2024.10691788

[70] Shang Liu, Yao Lu, Wenji Fang, Mengming Li, and Zhiyao Xie. 2025. OpenLLM-RTL: Open Dataset and Benchmark for LLM-Aided Design RTL Generation. arXiv:2503.15112. Retrieved from https://arxiv.org/abs/2503.15112

[71] Yi Liu, Changran Xu, Yunhao Zhou, Zeju Li, and Qiang Xu. 2025. DeepRTL: Bridging verilog understanding and generation with a unified representation model. In *Proceedings of the 13th International Conference on Learning Representations*. OpenReview.net. Retrieved from https://openreview.net/forum?id=2hcfoCHKoB

[72] Yi Liu, Hongji Zhang, Yunhao Zhou, Zhengyuan Shi, Changran Xu, and Qiang Xu. 2025. DeepRTL2: A versatile model for RTL-related tasks. In *Findings of the Association for Computational Linguistics*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, 6485–6500. Retrieved from https://aclanthology.org/2025.findings-acl.336/

[73] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. Starcoder 2 and the stack v2: The next generation. arXiv:2402.19173. Retrieved from https://arxiv.org/abs/2402.19173

[74] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2024. RTLLM: An open-source benchmark for design RTL generation with large language model. In *Proceedings of the 29th Asia and South Pacific Design Automation Conference*. IEEE, 722–727. DOI: https://doi.org/10.1109/ASP-DAC58780.2024.10473904

[75] Lakshmi Likhitha Mankali, Jitendra Bhandari, Manaar Alam, Ramesh Karri, Michail Maniatakos, Ozgur Sinanoglu, and Johann Knechtel. 2025. RTL-Breaker: Assessing the security of LLMs against backdoor attacks on HDL code generation. In *Proceedings of the Design, Automation and Test in Europe Conference*. IEEE, 1–7. DOI: https://doi.org/10.23919/DATE64628.2025.10993260

[76] Zhendong Mi, Renming Zheng, Haowen Zhong, Yue Sun, Seth Kneeland, Sayan Moitra, Ken Kutzer, and Zhaozhuo Xu Shaoyi Huang. 2024. CoopetitiveV: Leveraging LLM-powered Coopetitive Multi-Agent Prompting for High-quality Verilog Generation. arXiv:2412.11014. Retrieved from https://arxiv.org/abs/2412.11014

[77] Kyungjun Min, Seonghyeon Park, Hyeonwoo Park, Jinoh Cho, and Seokhyeong Kang. 2025. Improving LLM-based verilog code generation with data augmentation and RL. In *Proceedings of the Design, Automation and Test in Europe Conference*. IEEE, 1–7. DOI: https://doi.org/10.23919/DATE64628.2025.10992897

[78] Bardia Nadimi, Ghali Omar Boutaib, and Hao Zheng. 2025. PyraNet: A multi-layered hierarchical dataset for verilog. In *Proceedings of the 62nd ACM/IEEE Design Automation Conference*. IEEE, 1–7. DOI: https://doi.org/10.1109/DAC63849.2025.11133406

[79] Bardia Nadimi, Ghali Omar Boutaib, and Hao Zheng. 2025. VeriMind: Agentic LLM for Automated Verilog Generation with a Novel Evaluation Metric. arXiv:2503.16514. Retrieved from https://arxiv.org/abs/2503.16514

[80] Bardia Nadimi and Hao Zheng. 2024. A multi-expert large language model architecture for verilog code generation. In *Proceedings of the 2024 IEEE LLM Aided Design Workshop*. 1–5. DOI: https://doi.org/10.1109/LAD62341.2024.10691683

[81] Andre Nakkab, Sai Qian Zhang, Ramesh Karri, and Siddharth Garg. 2024. Rome was not built in a single step: Hierarchical prompting for LLM-based chip design. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*, Hussam Amrouch, Jiang Hu, Siddharth Garg, and Yibo Lin (Eds.). ACM, 26:1–26:11. DOI: https://doi.org/10.1145/3670474.3685964

[82] Erik Nijkamp, Hiroaki Hayashi, Caiming Xiong, Silvio Savarese, and Yingbo Zhou. 2023. CodeGen2: Lessons for Training LLMs on Programming and Natural Languages. arXiv:2305.02309. Retrieved from https://arxiv.org/abs/2305.02309

[83] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An open large language model for code with multi-turn program synthesis. In *Proceedings of the 11th International Conference on Learning Representations*. OpenReview.net. Retrieved from https://openreview.net/forum?id=iaYcJKpY2B_

[84] Hammond Pearce, Benjamin Tan, and Ramesh Karri. 2020. DAVE: Deriving automatically verilog from english. In *Proceedings of the ACM/IEEE Workshop on Machine Learning for CAD*, Ulf Schlichtmann, Raviv Gal, Hussam Amrouch, and Hai (Helen) Li (Eds.). ACM, 27–32. DOI: https://doi.org/10.1145/3380446.3430634

[85] Zehua Pei, Hui-Ling Zhen, Mingxuan Yuan, Yu Huang, and Bei Yu. 2024. BetterV: Controlled verilog generation with discriminative guidance. In *Proceedings of the 41st International Conference on Machine Learning*. OpenReview.net. Retrieved from https://openreview.net/forum?id=jKnW7r7de1

[86] Qiwei Peng, Yekun Chai, and Xuhong Li. 2024. HumanEval-XL: A multilingual code generation benchmark for cross-lingual natural language generalization. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, Nicoletta Calzolari, Min-Yen Kan, Véronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue (Eds.). ELRA and ICCL, 8383–8394. Retrieved from https://aclanthology.org/2024.lrec-main.735

[87] Nathaniel Pinckney, Christopher Batten, Mingjie Liu, Haoxing Ren, and Brucek Khailany. 2025. Revisiting VerilogEval: A year of improvements in large-language models for hardware code generation. *ACM Transactions on Design Automation of Electronic Systems* 30, 6 (2025), 20 pages. DOI: https://doi.org/10.1145/3718088

[88] Nathaniel Pinckney, Chenhui Deng, Chia-Tung Ho, Yun-Da Tsai, Mingjie Liu, Wenfei Zhou, Brucek Khailany, and Haoxing Ren. 2025. Comprehensive Verilog Design Problems: A Next-Generation Benchmark Dataset for Evaluating Large Language Models and Agents on RTL Design and Verification. arXiv:2506.14074. Retrieved from https://arxiv.org/abs/2506.14074

[89] Heng Ping, Shixuan Li, Peiyu Zhang, Anzhe Cheng, Shukai Duan, Nikos Kanakaris, Xiongye Xiao, Wei Yang, Shahin Nazarian, Andrei Irimia, and Paul Bogdan. 2025. HDLCoRe: A Training-Free Framework for Mitigating Hallucinations in LLM-Generated HDL. arXiv:2503.16528. Retrieved from https://arxiv.org/abs/2503.16528

[90] Suresh Purini, Siddhant Garg, Mudit Gaur, Sankalp Bhat, Sohan Mupparapu, and Arun Ravindran. 2025. ArchXBench: A complex digital systems benchmark suite for LLM driven RTL synthesis. In *Proceedings of the 7th ACM/IEEE Symposium on Machine Learning for CAD*. IEEE, 1–10. DOI: https://doi.org/10.1109/MLCAD65511.2025.11189156

[91] Haiyan Qin, Zhiwei Xie, Jingjing Li, Liangchen Li, Xiaotong Feng, Junzhan Liu, and Wang Kang. 2025. ReasoningV: Efficient Verilog Code Generation with Adaptive Hybrid Reasoning Model. arXiv:2504.14560. Retrieved from https://arxiv.org/abs/2504.14560

[92] Sriram Ranga, Rui Mao, Debjyoti Bhattacharjee, Erik Cambria, and Anupam Chattopadhyay. 2024. RTL Agent: An agent-based approach for functionally correct HDL generation via LLMs. In *Proceedings of the 33rd IEEE Asian Test Symposium*. IEEE, 1–6. DOI: https://doi.org/10.1109/ATS64447.2024.10915277

[93] Prithwish Basu Roy, Akashdeep Saha, Manaar Alam, Johann Knechtel, Michail Maniatakos, Ozgur Sinanoglu, and Ramesh Karri. 2025. Veritas: Deterministic Verilog Code Synthesis from LLM-Generated Conjunctive Normal Form. arXiv:2506.00005. Retrieved from https://arxiv.org/abs/2506.00005

[94] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. arXiv:2308.12950. Retrieved from https://arxiv.org/abs/2308.12950

[95] Humza Sami, Mubashir ul Islam, Samy Charas, Asav Gandhi, Pierre-Emmanuel Gaillardon, and Valerio Tenace. 2025. Nexus: A Lightweight and Scalable Multi-Agent Framework for Complex Tasks Automation. arXiv:2502.19091. Retrieved from https://arxiv.org/abs/2502.19091

[96] Wenhao Sun, Bing Li, Grace Li Zhang, Xunzhao Yin, Cheng Zhuo, and Ulf Schlichtmann. 2025. Paradigm-based automatic HDL code generation using LLMs. In *Proceedings of the 26th International Symposium on Quality Electronic Design*. IEEE, 1–8. DOI: https://doi.org/10.1109/ISQED65160.2025.11014391

[97] Jinwei Tang, Jiayin Qin, Kiran Thorat, Chen Tian, Yu Cao, Yang Zhao, and Caiwen Ding. 2024. HiVeGen - Hierarchical LLM-based Verilog Generation for Scalable Chip Design. arXiv:2412.05393. Retrieved from https://arxiv.org/abs/2412.05393

[98] Kimia Tasnia, Alexander Garcia, Tasnuva Farheen, and Sazadur Rahman. 2025. VeriOpt: PPA-Aware High-Quality Verilog Generation via Multi-Role LLMs. arXiv:2507.14776. Retrieved from https://arxiv.org/abs/2507.14776

[99] Llama Team. 2024. The Llama 3 Herd of Models. arXiv:2407.21783. Retrieved from https://arxiv.org/abs/2407.21783

[100] Shailja Thakur, Baleegh Ahmad, Zhenxing Fan, Hammond Pearce, Benjamin Tan, Ramesh Karri, Brendan Dolan-Gavitt, and Siddharth Garg. 2023. Benchmarking large language models for automated verilog RTL code generation. In *Proceedings of the Design, Automation and Test in Europe Conference and Exhibition*. IEEE, 1–6. DOI: https://doi.org/10.23919/DATE56975.2023.10137086

[101] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. VeriGen: A large language model for verilog code generation. *ACM Trans. Design Autom. Electr. Syst.* 29, 3 (2024), 46:1–46:31. DOI: https://doi.org/10.1145/3643681

[102] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2023. AutoChip: Automating HDL Generation Using LLM Feedback. arXiv:2311.04887. Retrieved from https://arxiv.org/abs/2311.04887

[103] Kiran Thorat, Jiahui Zhao, Yaotian Liu, Amit Hasan, Hongwu Peng, Xi Xie, Bin Lei, and Caiwen Ding. 2025. LLM-VeriPPA: Power, performance, and area optimization aware verilog code generation with large language models. In *Proceedings of the 7th ACM/IEEE Symposium on Machine Learning for CAD*. IEEE, 1–7. DOI: https://doi.org/10.1109/MLCAD65511.2025.11189158

[104] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971. Retrieved from https://arxiv.org/abs/2302.13971

[105] Yunda Tsai, Mingjie Liu, and Haoxing Ren. 2024. RTLFixer: Automatically fixing RTL syntax errors with large language model. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*, Vivek De (Ed.). ACM, 53:1–53:6. DOI: https://doi.org/10.1145/3649329.3657353

[106] Mubashir ul Islam, Humza Sami, Pierre-Emmanuel Gaillardon, and Valerio Tenace. 2024. AIvril: AI-Driven RTL Generation With Verification In-The-Loop. arXiv:2306.11644. Retrieved from https://arxiv.org/abs/2306.11644

[107] Paola Vitolo, George Psaltakis, Michael A. Tomlinson, Gian Domenico Licciardo, and Andreas G. Andreou. 2024. Natural language to verilog: Design of a recurrent spiking neural network using large language models and ChatGPT. In *Proceedings of the International Conference on Neuromorphic Systems*. IEEE, 110–116. DOI: https://doi.org/10.1109/ICONS62911.2024.00024

[108] Gwok-Waa Wan, Yubo Wang, Sam-Zaak Wong, Jia Xiong, Qixiang Chen, Jingyi Zhang, Mingchi Zhang, Tao Ni, Mengnv Xing, Yusheng Hua, et al. 2025. GenBen: A generative benchmark for LLM-aided design. In *Arxiv*.

[109] Bowei Wang, Qi Xiong, Zeqing Xiang, Lei Wang, and Renzhi Chen. 2025. Rtlsquad: Multi-agent based interpretable rtl design. arXiv:2501.05470. Retrieved from https://arxiv.org/abs/2501.05470

[110] Jinghua Wang, Lily Jiaxin Wan, Sanjana Pingali, Scott Smith, Manvi Jha, Shalini Sivakumar, Xing Zhao, Kaiwen Cao, and Deming Chen. 2025. OpenRTLSet: A fully open-source dataset for large language model-based verilog module design. In *Proceedings of the 2025 IEEE International Conference on LLM-Aided Design*. 212–218. DOI: https://doi.org/10.1109/ICLAD65226.2025.00038

[111] Kun Wang, Kaiyan Chang, Mengdi Wang, Xingqi Zou, Haobo Xu, Yinhe Han, and Ying Wang. 2025. RTLMarker: Protecting LLM-generated RTL copyright via a hardware watermarking framework. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, Yuichi Nakamura and Yu Wang (Eds.). ACM, 808–813. DOI: https://doi.org/10.1145/3658617.3697774

[112] Ning Wang, Bingkun Yao, Jie Zhou, Yuchen Hu, Xi Wang, Nan Guan, and Zhe Jiang. 2025. Insights from Verification: Training a Verilog Generation LLM with Reinforcement Learning with Testbench Feedback. arXiv:2504.15804. Retrieved from https://arxiv.org/abs/2504.15804

[113] Ning Wang, Bingkun Yao, Jie Zhou, Yuchen Hu, Xi Wang, Zhe Jiang, and Nan Guan. 2025. Large language model for verilog generation with code-structure-guided reinforcement learning. In *Proceedings of the 2025 IEEE International Conference on LLM-Aided Design*. 164–170. DOI: https://doi.org/10.1109/ICLAD65226.2025.00025

[114] Yue Wang, Hung Le, Akhilesh Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. CodeT5+: Open code large language models for code understanding and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* Association for Computational Linguistics, 1069–1088. DOI: https://doi.org/10.18653/V1/2023.EMNLP-MAIN.68

[115] Yiting Wang, Guoheng Sun, Wanghao Ye, Gang Qu, and Ang Li. 2025. VeriReason: Reinforcement Learning with Testbench Feedback for Reasoning-Enhanced Verilog Generation. arXiv:2505.11849. Retrieved from https://arxiv.org/abs/2505.11849

[116] Zeng Wang, Minghao Shao, Jitendra Bhandari, Likhitha Mankali, Ramesh Karri, Ozgur Sinanoglu, Muhammad Shafique, and Johann Knechtel. 2025. VeriContaminated: Assessing LLM-Driven Verilog Coding for Data Contamination. arXiv:2503.13572. Retrieved from https://arxiv.org/abs/2503.13572

[117] Zeng Wang, Minghao Shao, Rupesh Raj Karn, Likhitha Mankali, Jitendra Bhandari, Ramesh Karri, Ozgur Sinanoglu, Muhammad Shafique, and Johann Knechtel. 2025. SALAD: Systematic Assessment of Machine Unlearing on LLM-Aided Hardware Design. arXiv:2506.02089. Retrieved from https://arxiv.org/abs/2506.02089

[118] Anjiang Wei, Huanmi Tan, Tarun Suresh, Daniel Mendoza, Thiago S. F. X. Teixeira, Ke Wang, Caroline Trippel, and Alex Aiken. 2025. VeriCoder: Enhancing LLM-Based RTL Code Generation through Functional Correctness Validation. arXiv:2504.15659. Retrieved from https://arxiv.org/abs/2504.15659

[119] Yangbo Wei, Zhen Huang, Huang Li, Wei W. Xing, Ting-Jung Lin, and Lei He. 2025. VFlow: Discovering Optimal Agentic Workflows for Verilog Generation. arXiv:2504.03723. Retrieved from https://arxiv.org/abs/2504.03723

[120] Claes Wohlin. 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, Martin J. Shepperd, Tracy Hall, and Ingunn Myrtveit (Eds.). ACM, 38:1–38:10. DOI: https://doi.org/10.1145/2601248.2601268

[121] Sam-Zaak Wong, Gwok-Waa Wan, Dongping Liu, and Xi Wang. 2024. VGV: Verilog generation using visual capabilities of multi-modal large language models. In *Proceedings of the 2024 IEEE LLM Aided Design Workshop*. IEEE, 1–5.

[122] Nan Wu, Yuan Xie, and Cong Hao. 2022. AI-assisted synthesis in next generation EDA: Promises, challenges, and prospects. In *Proceedings of the IEEE 40th International Conference on Computer Design*. IEEE, 207–214. DOI: https://doi.org/10.1109/ICCD56317.2022.00039

[123] Peiyang Wu, Nan Guo, Junliang Lv, Xiao Xiao, and Xiaochun Ye. 2025. RTLRepoCoder: Repository-Level RTL Code Completion through the Combination of Fine-Tuning and Retrieval Augmentation. arXiv:2504.08862. Retrieved from https://arxiv.org/abs/2504.08862

[124] Peiyang Wu, Nan Guo, Xiao Xiao, Wenming Li, Xiaochun Ye, and Dongrui Fan. 2025. ITERTL: An iterative framework for fine-tuning LLMs for RTL code generation. In *Proceedings of the IEEE International Symposium on Circuits and Systems*. IEEE, 1–5. DOI: https://doi.org/10.1109/ISCAS56072.2025.11044046

[125] Changran Xu, Yi Liu, Yunhao Zhou, Shan Huang, Ningyi Xu, and Qiang Xu. 2025. Speculative decoding for verilog: Speed and quality, all in one. In *Proceedings of the 62nd ACM/IEEE Design Automation Conference*. IEEE, 1–7. DOI: https://doi.org/10.1109/DAC63849.2025.11133030

[126] An Yang, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoyan Huang, Jiandong Jiang, Jianhong Tu, Jianwei Zhang, Jingren Zhou, Junyang Lin, Kai Dang, Kexin Yang, Le Yu, Mei Li, Minmin Sun, Qin Zhu, Rui Men, Tao He, Weijia Xu, Wenbiao Yin, Wenyuan Yu, Xiafei Qiu, Xingzhang Ren, Xinlong Yang, Yong Li, Zhiying Xu, and Zipeng Zhang. 2025. Qwen2.5-1M Technical Report. arXiv:2501.15383. Retrieved from https://arxiv.org/abs/2501.15383

[127] Guang Yang, Wei Zheng, Xiang Chen, Yifan Sun, Fengji Zhang, and Terry Yue Zhuo. 2025. The Cream Rises to the Top: Efficient Reranking Method for Verilog Code Generation. arXiv:2509.20215. Retrieved from https://arxiv.org/abs/2509.20215

[128] Lanxin Yang, He Zhang, Haifeng Shen, Xin Huang, Xin Zhou, Guoping Rong, and Dong Shao. 2021. Quality assessment in systematic literature reviews: A software engineering perspective. *Inf. Softw. Technol.* 130 (2021), 106397. DOI: https://doi.org/10.1016/J.INFSOF.2020.106397

[129] Yiyao Yang, Fu Teng, Pengju Liu, Mengnan Qi, Chenyang Lv, Ji Li, Xuhong Zhang, and Zhezhi He. 2025. HaVen: Hallucination-mitigated LLM for verilog code generation aligned with HDL engineers. In *Proceedings of the Design, Automation and Test in Europe Conference*. IEEE, 1–7. DOI: https://doi.org/10.23919/DATE64628.2025.10993072

[130] Jianmin Ye, Tianyang Liu, Qi Tian, Shengchu Su, Zhe Jiang, and Xi Wang. 2025. ChatModel: Automating Reference Model Design and Verification with LLMs. arXiv:2506.15066. Retrieved from https://arxiv.org/abs/2506.15066

[131] Patrick Yubeaton, Andre Nakkab, Weihua Xiao, Luca Collini, Ramesh Karri, Chinmay Hegde, and Siddharth Garg. 2025. VeriThoughts: Enabling Automated Verilog Code Generation using Reasoning and Formal Verification. arXiv:2505.20302. Retrieved from https://arxiv.org/abs/2505.20302

[132] Gaoche Zhang, Dingyang Zou, Kairui Sun, Zhihuan Chen, Meiqi Wang, and Zhongfeng Wang. 2025. GEMMV: An LLM-based automated performance-aware framework for GEMM verilog generation. *IEEE J. Emerg. Sel. Topics Circuits Syst.* 15, 2 (2025), 325–336. DOI: https://doi.org/10.1109/JETCAS.2025.3568712

[133] He Zhang, Muhammad Ali Babar, and Paolo Tell. 2011. Identifying relevant studies in software engineering. *Inf. Softw. Technol.* 53, 6 (2011), 625–637. DOI: https://doi.org/10.1016/J.INFSOF.2010.12.010

[134] Jiazheng Zhang, Cheng Liu, and Huawei Li. 2025. Understanding and Mitigating Errors of LLM-Generated RTL Code. arXiv:2508.05266. Retrieved from https://arxiv.org/abs/2508.05266

[135] Yongan Zhang, Zhongzhi Yu, Yonggan Fu, Cheng Wan, and Yingyan Celine Lin. 2024. MG-Verilog: Multi-grained Dataset Towards Enhanced LLM-assisted Verilog Generation. arXiv:2407.01910. Retrieved from https://arxiv.org/abs/2407.01910

[136] Heri Zhao, Jeffrey Hui, Joshua Howland, Nam Nguyen, Siqi Zuo, Andrea Hu, Christopher A. Choquette-Choo, Jingyue Shen, Joe Kelley, Kshitij Bansal, Luke Vilnis, Mateo Wirth, Paul Michel, Peter Choy, Pratik Joshi, Ravin Kumar, Sarmad Hashmi, Shubham Agrawal, Zhitao Gong, Jane Fine, Tris Warkentin, Ale Jakse Hartman, Bin Ni, Kathy Korevec, Kelly Schaefer, and Scott Huffman. 2024. CodeGemma: Open Code Models Based on Gemma. arXiv:2406.11409. Retrieved from https://arxiv.org/abs/2406.11409

[137] Jiayu Zhao and Song Chen. 2025. VeriGRAG: Enhancing LLM-Based Verilog Code Generation with Structure-Aware Soft Prompts. arXiv:2510.15914. Retrieved from https://arxiv.org/abs/2510.15914

[138] Qianhui Zhao, Fang Liu, Xiao Long, Chengru Wu, and Li Zhang. 2025. On the applicability of code language models to scientific computing programs. *IEEE Trans. Software Eng.* 51, 6 (2025), 1685–1701. DOI: https://doi.org/10.1109/TSE.2025.3564599

[139] Yang Zhao, Di Huang, Chongxiao Li, Pengwei Jin, Muxin Song, Yinan Xu, Ziyuan Nan, Mingju Gao, Tianyun Ma, Lei Qi, Yansong Pan, Zhenxing Zhang, Rui Zhang, Xishan Zhang, Zidong Du, Qi Guo, and Xing Hu. 2025. CodeV: Empowering LLMs with HDL generation through multi-level summarization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 45, 4 (2025), 1893–1906. DOI: https://doi.org/10.1109/TCAD.2025.3604320

[140] Yujie Zhao, Hejia Zhang, Hanxian Huang, Zhongming Yu, and Jishen Zhao. 2025. MAGE: A multi-agent engine for automated RTL code generation. In *Proceedings of the 62nd ACM/IEEE Design Automation Conference*. IEEE, 1–7. DOI: https://doi.org/10.1109/DAC63849.2025.11133191

[141] Zhuorui Zhao, Ruidi Qiu, Ing-Chao Lin, Grace Li Zhang, Bing Li, and Ulf Schlichtmann. 2025. VRank: Enhancing verilog code generation from large language models via self-consistency. In *Proceedings of the 26th International Symposium on Quality Electronic Design*. IEEE, 1–7. DOI: https://doi.org/10.1109/ISQED65160.2025.11014398

[142] Yaoyu Zhu, Di Huang, Han-Qi Lyu, Xiaoyun Zhang, Chongxiao Li, Wenxuan Shi, Yutong Wu, Jianan Mu, Jinghua Wang, Yang Zhao, Pengwei Jin, Shuyao Cheng, Shengwen Liang, Xishan Zhang, Rui Zhang, Zidong Du, Qi Guo, Xing Hu, and Yunji Chen. 2025. CodeV-R1: Reasoning-Enhanced Verilog Generation. arXiv:2505.24183. Retrieved from https://arxiv.org/abs/2505.24183

Received 19 November 2025; revised 3 July 2026; accepted 13 August 2026